Mapping LLM-agent reliability under chaos: a cross-architecture study of structured recovery playbooks

Chaos Playbook Lab — Working Paper 1

Alberto Martínez Zurita · Alessandro Russo

Chaos Playbook Lab

First version: 19 May 2026 · This version: October 2026 (v1.1)

Code, dataset and figures: github.com/alberto-martinez-zurita/chaos-playbook-lab

Abstract

Large language model (LLM) agents are increasingly used as autonomous executors of multi-step business workflows, yet their reliability under *operational chaos*—transient tool failures such as timeouts, rate limits and server errors—remains poorly characterised. This matters beyond engineering: the EU AI Act requires high-risk AI systems to be resilient to "errors, faults or inconsistencies" arising from their interaction with other systems (Art. 15(4)), but prescribes no method to measure it. We use **chaos engineering as a scientific instrument**—controlled, seeded fault injection at the agent–tool boundary—and report a controlled study across **eleven open and frontier LLMs** (0.8 B to 355 B parameters; dense, Per-Layer-Embedding, hybrid Mamba2–Transformer and Mixture-of-Experts architectures), three agent–tool interface styles emulated in-process (HTTP, the Model Context Protocol (MCP) and the Agent-to-Agent protocol (A2A)) and ten injected failure rates p (the per-call probability that a tool call is replaced by an explicit, transient error) from 0 % to 30 %.

For each model we run a paired A/B comparison between an unguided agent (`nopb`) and the same model executing under a short structured **recovery playbook** (`pb`) that we designed for this study (69 words for HTTP), totalling 128,000 active agent runs. The primary outcome is the **success rate** (SR, the fraction of runs that complete the four-step workflow) and its difference Δ(pb–nopb) in percentage points; secondary outcomes are tokens, LLM calls and wall time per run. Three findings emerge: **(i)** The playbook closes most of the chaos-induced failure gap: over the **functioning roster** (n = 9 models, excluding a capacity-floor model and a structural-rescue case), the median Δ at 30 % chaos is **+43.5 percentage points** (bootstrap 95 % confidence interval (CI) over models [40.0, 55.0]); in paired exact McNemar tests with Holm correction the effect is significant in every functioning-roster HTTP cell at failure rates of 5 % and above, and no model is harmed at any rate under the transient faults we inject. Much of the effect is explained by retry behaviour: without the playbook several models rarely retry a failed call, and the playbook induces them to do so. **(ii)** The benefit depends on capability and on the chaos rate: at 30 % chaos the largest gains are in the mid-range (up to +67 pp at 14 B) and the frontier model gains less (+40 pp), while at 10 % chaos the gain is not monotonic in model size. We report this as an observed pattern, not a formally tested non-monotonicity. **(iii)** The cost is

modest: 13 %–69 % more tokens per run (mean ≈ 42 %) and +0.57 LLM calls on average; wall time is model-dependent (−20 % to +43 %).

We also document a *capacity floor* (qwen3.5 0.8 B fails the task at 0 % chaos in both arms) and a *structural-rescue artefact* (qwen3.5 2 B succeeds at 0 % chaos only with the playbook, which supplies missing output structure). The chaos rig, playbook, the 128,800-run dataset and the analysis scripts are released as open source.

Keywords: chaos engineering, LLM agents, reliability, recovery playbook, MCP, A2A.

1 Introduction

1.1 Why chaos, why now

Agentic large language model (LLM) stacks—a model orchestrating a sequence of tool calls against external systems—are no longer a research curiosity. They underpin retrieval-augmented assistants, integrated development environment (IDE) copilots, customer-support bots, extract-transform-load (ETL) repair jobs and a growing class of business-process automation. Yet the deployment story for these stacks still rests on two implicit assumptions:

1. The model is *good enough* to recover from the kind of imperfect tool behaviour that real production environments exhibit (timeouts, malformed JSON, half-applied writes, race conditions, transient HTTP 5xx).
2. Standard benchmarks (GSM8K [20], MMLU [19], HumanEval [21], agentic suites such as τ-Bench [18] or SWE-Bench [13]) are *informative enough* about that recovery behaviour to inform model-selection and architecture decisions.

We argue that neither assumption holds in practice. Standard benchmarks measure an agent's behaviour on a happy path. Reliability under chaos—partial failures, redelivery, drift between protocol layers—is a qualitatively different surface. It determines whether a deployed agent compounds errors silently or recovers gracefully, and it is the surface that a site reliability engineer (SRE) actually watches.

Two terms are used throughout. **Operational chaos** is the phenomenon: the transient failures an agent meets in production when its tools are slow, rate-limited or briefly unavailable. **Chaos engineering** [1, 3] is the practice, originating at Netflix, of injecting such failures deliberately and under control to observe how a system behaves. Our contribution is to **treat chaos engineering as a scientific instrument**, not a production-hardening tool, and to use it to expose the reliability landscape of LLM agents.

The question also has a regulatory side. The EU AI Act [33] requires providers of high-risk AI systems to ensure accuracy, robustness and cybersecurity (Art. 15), including resilience to "errors, faults or inconsistencies" arising from interaction with other systems (Art. 15(4)), and asks for the development of "benchmarks and measurement methodologies" (Art. 15(2)) without prescribing them. Evidence of how an agent behaves when its tools misbehave is exactly that kind of measurement, and it is largely absent from current practice (§2.4, §5.7).

1.2 The instrument

We build a transparent, seeded chaos rig (released as `chaos-engine`) that injects a controlled rate of faults into the path between the LLM agent and its tools. It re-architects an earlier prototype into a protocol-agnostic A/B rig (see Acknowledgements). The faults are intentionally simple and explicitly signalled—HTTP error codes (5xx, 408/504 timeouts, 429), typed errors of the Model Context Protocol (MCP: `timeout`, `rate_limited`, `internal_error`) and typed errors of the Agent-to-Agent protocol (A2A: `timeout`, `task_rejected`, `partial_failure`)—and the injection mechanism is the same for all protocols: each tool call is replaced by a fault with probability p, and the fault type is drawn uniformly from the rig's set. A single knob—the **failure rate** p—goes from 0 % to 30 % per tool call. Around the rig we run a fixed multi-step task and an A/B comparison between two agents:

- `nopb` **(unguided)**: the model is told what tools exist and asked to complete the task. It receives no recovery instructions.
- `pb` **(playbook)**: the model also receives a short structured **recovery playbook** that we designed for this study—a 69-word instruction block (HTTP version) stating, for each error type the rig can inject, whether and how to retry and when to give up.

Both arms run on the *same model*, the *same prompt skeleton* and the *same seed for each run index*. Differences in success rate, token cost and wall time are therefore attributable to the playbook and to the retries it elicits.

1.3 What this paper claims

We apply this instrument to **eleven LLMs** spanning nearly three orders of magnitude (0.8 B to 355 B total parameters), four architecture types (dense, Per-Layer-Embedding, hybrid Mamba2–Transformer and Mixture-of-Experts), one think/nothink toggle (GLM 4.5) plus two models that reason by default, and three agent–tool interface styles (HTTP, MCP and A2A, emulated in-process). The full dataset is **128,000 active agent runs** (plus 800 appendix runs from an excluded model, §3.6), all released as raw CSVs alongside this paper.

Four contributions:

1. **An open, reproducible instrument and dataset.** A seeded chaos rig, a recovery playbook per protocol and a 128,800-run dataset from which every figure and table in this paper can be regenerated.
2. **The playbook closes most of the chaos-induced gap.** Over the **functioning roster** (n = 9: the active roster minus the capacity-floor qwen3.5 0.8 B and the structural-rescue qwen3.5 2 B, §5.3), the median improvement at 30 % chaos is **+43.5 pp**; across the full active roster the range is +0 pp (qwen3.5 0.8 B) to +67 pp (Ministral 14 B). In paired exact tests with multiple-comparison correction the effect is significant in every functioning-roster HTTP cell at failure rates of 5 % and above, and no model is harmed at any rate under the transient faults we inject. A large part of the gain comes from inducing retries in models that otherwise rarely retry (§5.5 xi).
3. **The benefit depends on capability and on the chaos rate.** At 30 % chaos the mid-range models gain the most (largest at 14 B) and the 355 B frontier model gains less because its unguided baseline is already high; at 10 % chaos the gain is not monotonic in model size. We characterise this pattern and its limits rather than claiming a formal law.

4. **The cost is bounded and predictable.** The playbook adds 13 %–69 % more tokens per run (mean ≈ 42 %) and +0.57 LLM calls on average (range +0.25 to +0.98). Wall time is model-dependent: some models run *faster* with the playbook (qwen3.5 4 B: −20 %; GLM 4.5 nothink: −7 %), others moderately slower (Ministral 14 B: +27 %; Gemma 4 26 B: +43 %).

Two negative findings deserve attention:

- A **capacity floor**: qwen3.5 0.8 B fails the task at 0 % chaos in both arms. The playbook cannot rescue a model that cannot follow the original instructions.
- A **structural-rescue artefact**: qwen3.5 2 B's `nopb` arm scores 0 % at chaos = 0 % because the model cannot emit valid structured output for the task without explicit guidance. Its +56.5 pp gap at chaos = 10 % is largely the playbook fixing a *format* problem rather than a *chaos* problem. We discuss this caveat in §5.3.

A twelfth model (`gpt-oss-20b`) is excluded from comparative analysis because of a decoder-level failure at high chaos rates; details and rationale in §3.6.

1.4 Roadmap

§2 places the work in context. §3 describes the task, the playbook, the chaos rig and the model roster. §4 is the empirical core (Figs 1–8). §5 discusses caveats, threats to validity and practical implications. §6 closes with future directions. All figures and tables can be regenerated from the released data by running `python scripts/generate_paper_figures.py`.

2 Related work

This paper draws on three research communities: chaos engineering and fault injection for distributed systems (§2.1), LLM-agent benchmarking and robustness (§2.2), and the study of how capability changes with model scale, distillation and reasoning (§2.3). A fourth thread, regulatory requirements for robustness evidence, motivates the work (§2.4).

2.1 Chaos engineering and fault injection for distributed systems

Community: site reliability engineering and distributed-systems research. Chaos engineering originated as a production-hardening practice at Netflix with Chaos Monkey [2] and was formalised in the *Principles of Chaos Engineering* [3] and the IEEE Software treatment [1]. Subsequent work generalised the approach to controlled fault injection in production-grade platforms [6, 7, 8] and to formal models of failure such as Lineage-Driven Fault Injection [4] and its industrial application [5]. The standard target is *infrastructure*: kill a pod, throttle a network link, pause a process, return 5xx with probability p.

Our work inherits the philosophy—*inject controlled faults and observe*—and the methodological discipline (seeded injection, paired control runs, bounded blast radius). It departs in the *target*: we inject at the **agent ↔ tool boundary** of an LLM stack, not at the infrastructure level. The fault model is consequently different: transient failures surfaced through the tool-call channel itself—HTTP 5xx and timeout codes, typed MCP/A2A errors such as `rate_limited` or `task_rejected`—the failure modes a real LLM agent encounters when its tools are flaky.

A small set of **concurrent** works explores this direction: ReliabilityBench [37] injects timeouts, rate limits, partial responses and schema drift at the tool boundary across two frontier APIs (Gemini 2.0 Flash, GPT-4o) over 1,280 episodes; Hell or High Water [38] benchmarks agent recovery when functions disappear or fail mid-trajectory; AgentNoiseBench [39] categorises noise into user-noise and tool-noise in an automated pipeline. More recent work extends this direction: ToolMaze [43] benchmarks replanning and anomaly recovery across workflow topologies and a taxonomy of explicit and implicit perturbations, and AgentCheck [44] turns an MCP server into a fault-injection surface to reproduce failures and test mitigations such as retries. Our contribution differs on four axes: (i) **scale** (128,000 runs vs 1,280 episodes in ReliabilityBench), (ii) **roster breadth** (eleven models spanning 0.8 B to 355 B vs two frontier APIs), (iii) **a controlled, paired A/B comparison with a structured recovery playbook across many models** (AgentCheck tests retry mitigations on individual agents, but not as a controlled comparison across models), and (iv) **coverage of three agent–tool interface styles** (HTTP, MCP and A2A, emulated in-process).

2.2 LLM-agent benchmarks and robustness

Community: machine-learning evaluation and agent research. The LLM-agent literature has produced an expanding family of benchmarks: **SWE-Bench** [13] (software engineering), **AgentBench** [14], **WebArena** [15], **OSWorld** [17], **τ-Bench** [18] (multi-turn business-process completion) and **GAIA** [16]. These suites measure end-to-end task success on golden trajectories in deterministic environments. They have driven real progress in tool use and planning, but by construction they say little about behaviour under *operational* chaos: the environments are clean, the tools always work, and the only source of failure is the model itself.

A second strand examines *robustness* to perturbed or adversarial inputs, such as jailbreak red-teaming and prompt-injection attacks [34]. These manipulate the *input* the model receives, mimicking a careless or malicious user or a poisoned document. Our axis is complementary: we keep prompts fixed and inject faults into the *tool responses* the agent receives mid-trajectory. This mimics what production infrastructure does to an agent—an API that times out, a rate limiter that pushes back, a service that is briefly down—failures that are frequent, benign in intent and invisible to input-side robustness tests.

A third strand adds model-side recovery behaviour: ReAct [9] interleaves reasoning and acting, and Reflexion [10] adds verbal self-reflection between attempts; relatedly, Tree of Thoughts [11] performs deliberate search with self-evaluation and backtracking. These works make the *model* invent recovery behaviour. We ask the complementary question: given a fixed model, how much of the recovery gap can a *short, externally provided* recovery playbook close? Because the model is held fixed in each comparison, the effect is comparable across architectures.

2.3 Scale, distillation and reasoning

Community: language-model scaling and training research. A large body of work studies how capability changes with scale and architecture—compute-optimal training [22], knowledge distillation [23], sparse Mixture-of-Experts models such as Mixtral [24]—and the literature on inference-time reasoning (chain-of-thought [12], deliberation and "thinking" toggles in current model families [25, 26, 27]). These findings are established on *clean* benchmarks.

We vary the same axes—total and active parameters, architecture, distilled vs original, think vs nothink—under chaos, and find a different picture:

- The playbook's benefit is not monotonic in scale: at high chaos the mid-range models gain the most and the frontier model gains less, whereas capability on clean benchmarks usually grows with scale. A similar inverted-U has been reported for prompting strategies on reasoning tasks [40].
- In our **distillation pair** (qwen3.5 9 B vs a community-distilled 9 B variant), the distilled model scores above the original on the unguided arm at every chaos rate (+1 pp at chaos = 0 % to +13 pp at chaos = 30 %), although only the differences at 10 % and 30 % chaos are individually significant.
- The think/nothink toggle on GLM 4.5 has little effect on flat HTTP tools but lowers unguided success on protocols with typed errors (MCP, A2A).

We treat these as empirical observations, not as theory.

2.4 Regulatory requirements for robustness evidence

The EU AI Act [33] requires *providers* of high-risk AI systems to comply with requirements on risk management (Art. 9), data governance (Art. 10), human oversight (Art. 14) and accuracy, robustness and cybersecurity (Art. 15). Article 15(4) asks that such systems be "as resilient as possible regarding errors, faults or inconsistencies" arising from their interaction with natural persons or other systems; providers of general-purpose AI models *with systemic risk* must additionally perform model evaluation including adversarial testing (Art. 55(1)(a)). The Act encourages the development of "benchmarks and measurement methodologies" (Art. 15(2)) without prescribing them, a gap analysed in [42]. Proposed assurance frameworks focus on documentation rather than active testing (e.g. the ontology and assurance-case approach of [41]). Evidence of *what an agent does when its tools misbehave* is largely absent from both guidance and practice. We present the rig and the released dataset as a candidate measurement methodology for that gap (§5.7).

2.5 What this paper is *not*

This is **not** a paper about a new model, a new training recipe or a new agent framework. The agent framework is deliberately minimal. The contribution is the *measurement methodology* and the *empirical map* it produces; the instrument and the data are the artefact.

3 Methodology

3.1 The task

The system under test is an LLM agent executing a deterministic four-step business workflow modelled on the OpenAPI Petstore reference:

#	Step	Endpoint (HTTP)	Purpose
1	`FIND_PRODUCT`	`GET /product/find`	Resolve a product by query string.
2	`CHECK_INVENTORY`	`GET /store/inventory`	Confirm stock for the resolved product.
3	`PLACE_ORDER`	`POST /store/order`	Place the order, receive an order id.
4	`CONFIRM_ORDER`	`PUT /order/confirm`	Verify the order was committed.

The four-step shape is intentionally minimal but realistic: each step depends on the previous one (the `order_id` from step 3 is a parameter to step 4), so a failed intermediate call derails the rest of the trajectory unless the agent recognises the failure and recovers before moving on.

A run is **successful** if and only if the agent completes all four steps in order—each call returning a non-error response—within the run's turn budget. Anything else—an injected fault the agent never recovers from, an explicit abort, an unparseable action, or an exhausted turn budget—is recorded as `failure`, together with the exact step at which the run failed (`failed_at`) and the steps completed before that (`steps_completed`). Success therefore measures **trajectory completion under chaos**: the oracle validates the call choreography, not the semantic consistency of every field the agent carries between steps. Parameter-chaining fidelity (e.g. reusing the `order_id` from step 3 in step 4) is instructed by the prompt and recorded in the released per-turn traces, but is not part of the success predicate; per-field state-tracking accuracy is flagged as trace-level follow-up (§5.5 ix). Both arms face the same oracle, so the A/B contrast is unaffected by this choice.

The agent loop. At each turn the model receives the system prompt (task description, available tools and, in the `pb` arm, the playbook) and a single harness-generated message summarising the previous step: the call made, its status (the tool result or the injected error) and the list of steps completed so far. The model does not see the earlier conversation; the workflow state is carried by this summary. The model answers with one action: a tool call, `complete` or `abort`. A retry is the model re-issuing the call that failed; it is counted in the `retries` column. The agent has no wait action, so the backoff delays mentioned in the playbook are not executed: retries are immediate. Each run has a turn budget of 15 turns (HTTP) or 12 turns (MCP and A2A); a run that exhausts it is a failure. The released repository also contains an LLM-free deterministic agent (`agents/deterministic.py`) that executes the workflow with a fixed retry strategy; it was not run as a baseline arm in this campaign (§5.5 xi).

3.2 The chaos rig

A small chaos proxy sits between the agent and the tool endpoints. Each tool call is faulted with probability p (the configured **failure rate**), independently of other calls; when a fault is triggered, the proxy replaces the tool response with an explicit, typed fault whose type is drawn uniformly at random from the rig's fault set. Two design decisions shape the fault model of this campaign. First, every injected fault is **visible**: the agent receives an error status plus a protocol-appropriate error message, never a silently corrupted payload—the study therefore measures *recovery from detectable failures*, not *detection of corrupted data*. Second, every injected fault is **transient by design**—a fault class for which retrying is a sensible strategy. The per-rig fault sets are:

- **HTTP rig**: the response is replaced by an error whose status code is drawn uniformly from {408, 429, 500, 502, 503, 504}, with a generic error body (`chaos/proxy.py`).
- **MCP rig** (`MCPFailureType` in the released code): `timeout`, `rate_limited`, `internal_error`, drawn uniformly (`chaos/mcp_proxy.py`).
- **A2A rig** (`A2AFailureType`): `timeout`, `task_rejected`, `partial_failure`, drawn uniformly (`chaos/a2a_proxy.py`).

The released engine defines a broader catalog—six MCP-typed and six A2A-typed faults (including permanent classes such as `tool_not_found`, `invalid_params`, `context_overflow`, `agent_not_found` and `authentication_error`) plus a payload-corruption library (drop a required field, swap a field's type, truncate the payload, return an empty body)—but this campaign deliberately injects only the transient

subset above. Permanent faults would contaminate the A/B comparison with unrecoverable cells, and silently corrupted payloads test detection rather than recovery; we flag silent-corruption chaos as future work (§6.2).

The proxy's random number generator is seeded per run with seed = 2026 + 1000·*k* + *j*, where *j* is the run index and *k* the position of the failure rate in the list of rates of a given launch of the runner. Both arms of a pair are launched together, so they share the seed and start from the same fault schedule; the seed is recorded in every CSV row and the same per-run seed is passed to the model API as its sampling seed. Most cells were launched one rate at a time (*k* = 0, seeds 2026 + *j*); some GLM 4.5 and Gemma 4 26 B MCP/A2A runs were collected in launches covering several rates, which gave them seed offsets of 1000·*k* (3026 + *j*, 5026 + *j*, …), and in a few of those cells the two arms were not launched together, so they are not paired run by run for every index. The paired tests of §3.7 therefore use only the HTTP cells, which are fully paired, and §5.5 (vii) uses the different offsets as a seed-robustness check. For a given seed, failure rate and sequence of tool calls, the fault sequence is reproducible. Two caveats apply. First, the *model's* output is not deterministic (sampling at temperature 0.2, §3.8, and runtime-dependent numerics), so re-running a cell on another machine reproduces the fault schedule but not necessarily every trajectory. Second, a retry is a new tool call that consumes a new random draw, so once the two arms take different actions their fault sequences diverge (§3.8). The three rigs are emulated in-process: no network transport and no official protocol SDK is involved. Three protocol-specific implementations of the rig exist:

- **HTTP**: REST-style calls (method, endpoint, parameters) answered by a mock Petstore inside the proxy, with HTTP status codes. In the HTTP rig the proxy is additionally wrapped by a **circuit breaker** that opens after three consecutive failed calls and, for the next 30 seconds of wall-clock time, answers every call with `503 Circuit Breaker Open`, then lets one probe call through; its state is reset at the start of each run. It is not present in the MCP and A2A rigs (see §5.5 xii).
- **MCP** (Model Context Protocol [31]): an emulated MCP tool interface—a tool catalogue with JSON input schemas and MCP-style results—with faults returned as the MCP-typed errors above.
- **A2A** (Agent-to-Agent [32]): each step is phrased as a task delegated to an emulated remote agent, with A2A-style task messages, and faults are returned as the A2A-typed errors above.

All three rigs share the same injection mechanism—a seeded Bernoulli draw per tool call—the same seed schedule, and the same transient-and-explicit fault design, so cross-protocol comparisons (Fig 5) share the fault mechanism: in every rig the agent faces a visible, retryable error at the same schedule points. The rigs still differ in error vocabulary, in the HTTP-only circuit breaker and in the turn budget (§5.5 xii). The fault *vocabularies* are intrinsic to each rig (you cannot inject a 504 timeout in A2A or a `task_rejected` in HTTP) and are documented as part of each rig's catalog rather than equated across protocols.

Playbook–rig coverage note. The HTTP playbook in §3.3 lists exactly the six status codes the HTTP rig can inject—408, 429, 500, 502, 503, 504—each present in the rig's fault catalog (`assets/knowledge_base/http_error_codes.json` in the released code). The MCP and A2A playbooks likewise cover exactly the typed faults their rigs inject (§3.2). Rig and playbook are therefore fully aligned on the recoverable fault set; no abort-only ("400/401/403"-style) rules appear in either, by design—the experiments inject only recoverable faults so that A/B differences attributable to the playbook reflect *recovery behaviour*, not *fail-fast routing*. See threats to validity (§5.5 iv) for the implication of having a single playbook content.

3.3 The two arms

For each `(model, protocol[, thinking-mode])` cell we evaluate two agents on identical inputs:

- `nopb` **(control)**: the model receives a system prompt that names the four steps and the available tools, plus the task instance. No further guidance.
- `pb` **(treatment)**: the same prompt prefix plus a structured **recovery playbook**, designed by the authors for this study. The HTTP playbook has 69 words and is reproduced verbatim below from `assets/prompts/http_playbook_guidance.txt`; MCP and A2A variants of comparable length use each protocol's error vocabulary:

```
Recovery playbook—when a step fails, apply these strategies:
  - HTTP 408, 504 → retry with linear backoff (max 3 retries, 2s delay)
  - HTTP 429 → wait 5s then retry (max 3 retries)
  - HTTP 500, 502, 503 → retry with exponential backoff (max 3, 1s base)
All errors are transient—always retry before giving up.
Always retry the exact same endpoint that failed.
Only abort after exhausting all retries for a given step.
```

The playbook lists exactly the status codes the rig can inject (cf. §3.2: 408, 429, 500, 502, 503, 504) and does *not* invent recovery semantics beyond what HTTP status conventions already imply. It is a forcing function: it asks the model to recognise standard signals and respond in a standard way.

The MCP and A2A playbooks (`assets/prompts/mcp_playbook_guidance.txt` and `assets/prompts/a2a_playbook_guidance.txt`) read:

```
Recovery playbook — when a tool call fails, apply these strategies:
  - timeout → retry same tool call with linear backoff (max 3 retries, 2s delay)
  - rate_limited → wait 5s then retry (max 3 retries)
  - internal_error → retry with exponential backoff (max 3, 1s base)
All errors are transient — always retry before giving up.
Always retry the exact same tool call that failed.
Only abort after exhausting all retries for a given step.
```

```
Recovery playbook — when a task delegation fails, apply these strategies:
  - timeout → retry with linear backoff (max 3 retries, 2s delay)
  - task_rejected → retry with exponential backoff (max 3, 1s base)
  - partial_failure → retry with linear backoff (max 3, 2s delay)
All errors are transient — always retry before giving up.
Always retry the exact same task delegation that failed.
Only abort after exhausting all retries for a given step.
```

3.4 Failure-rate sweep

We call a *sweep* the series of experiments that runs one `(model, protocol[, thinking])` configuration at each of ten failure rates:

```
0.00, 0.01, 0.02, 0.03, 0.05, 0.10, 0.15, 0.20, 0.25, 0.30
```

At each rate we run *N* paired runs per arm (the same task instance, the same seed per run index). All local-roster cells are *N* = 200 (ten models: the five qwen3.5 models, Gemma 4 E2B and E4B, Nemotron 3 Nano 4 B, Ministral 14 B across HTTP/MCP/A2A and Gemma 4 26 B across HTTP/MCP/A2A); the cloud frontier model (GLM 4.5) is *N* = 600 per arm, three times the local *N*, so that its six think/nothink × protocol cells are well powered. Total sample: **128,000 active runs**. Per-cell counts:

Runs summary (rows in CSVs).

Grand total: 128,800 runs across 416 non-empty cells.

Models marked with `*` are excluded from main comparative figures (Table 3).

Per model

Model	Protocols	Agent groups	Rates seen	Total rows	Min n/cell	Max n/cell
qwen35_0_8b	http	2	10	4000	200	200
qwen35_2b	http	2	10	4000	200	200
gemma4_e2b	http	2	10	4000	200	200
gemma4_e4b	http	2	10	4000	200	200
qwen35_4b	http	2	10	4000	200	200
nemotron_nano_4b	http	2	10	4000	200	200
qwen35_9b	http	2	10	4000	200	200
qwen35_9b_distilled	http	2	10	4000	200	200
ministral_14b	a2a, http, mcp	2	10	12000	200	200
gpt_oss_20b *	http	2	8	800	50	50
gemma4_26b	a2a, http, mcp	2	10	12000	200	200
glm45	a2a, http, mcp	4	10	72000	600	600

By protocol

Protocol	Runs	%
HTTP	64,800	50.3%
A2A	32,000	24.8%
MCP	32,000	24.8%
Total	**128,800**	100.0%

By reasoning mode (in protocol directory)

Reasoning	Runs	%
think	52,800	41.0%
n/a	40,000	31.1%
nothink	36,000	28.0%
Total	**128,800**	100.0%

By playbook arm

Arm	Runs	%
nopb	64,400	50.0%
pb	64,400	50.0%
Total	**128,800**	100.0%

By family

Family	Runs	%
glm4	72,000	55.9%
gemma4	20,000	15.5%
qwen3.5	20,000	15.5%
ministral3	12,000	9.3%
nemotron3	4,000	3.1%
gpt-oss	800	0.6%
Total	**128,800**	100.0%

By architecture

Architecture	Runs	%
moe	84,800	65.8%
dense	32,000	24.8%
ple	8,000	6.2%
hybrid_mamba2_transformer	4,000	3.1%
Total	**128,800**	100.0%

By total-params bucket

Total params	Runs	%
frontier (≥30B)	72,000	55.9%

Total params	Runs	%
5–26B	40,800	31.7%
2–4B	12,000	9.3%
sub-2B	4,000	3.1%
Total	**128,800**	100.0%

3.5 Model roster

The active roster comprises eleven models spanning nearly three orders of magnitude in total parameters and four architecture types.

Table 1 — Active model roster.

Model	Architecture	Total	Active	Reasoning	Provider	Runtime
qwen35_0_8b	dense	0.8B	0.8B	False	alibaba	local
qwen35_2b	dense	2B	2B	False	alibaba	local
gemma4_e2b	ple	5B	~2.3B	False	google	local
gemma4_e4b	ple	8B	~4.5B	False	google	local
qwen35_4b	dense	4B	4B	False	alibaba	local
nemotron_nano_4b	hybrid_mamba2_transformer	4B	4B	True	nvidia	local
qwen35_9b	dense	9B	9B	False	alibaba	local
qwen35_9b_distilled	dense	9B	9B	inherited	flipradio_community	local
ministral_14b	dense	14B	14B	True	mistral	local
gemma4_26b	moe	26B	4B	False	google	local
glm45	moe	355B	32B	optional	zhipu	cloud

In short:

- **Dense** (non-MoE): the qwen3.5 0.8 B, 2 B, 4 B and 9 B models [25] and a community-distilled qwen3.5 9 B variant, whose layers combine Gated DeltaNet linear attention with standard gated attention in a 3:1 ratio; and Ministral 14 B, a Transformer with reasoning [29].
- **Hybrid Mamba2–Transformer**: NVIDIA Nemotron 3 Nano 4 B (reasoning enabled by default; its reasoning toggle was not varied in this study), pruned and distilled from Nemotron Nano 9 B v2 via the Nemotron Elastic framework [28].
- **Per-Layer Embeddings (PLE)**: Gemma 4 E2B (~2.3 B effective of 5 B total) and Gemma 4 E4B (~4.5 B effective of 8 B total) [26]; each decoder layer has its own small token embedding, so the effective (compute) size is smaller than the total.
- **MoE**: Gemma 4 26 B (4 B active of 26 B total) and GLM 4.5 (32 B active of 355 B total).

The roster intentionally repeats the ~4 B active-parameter point with **four different architectures** (qwen3.5 4 B dense, Gemma 4 E4B PLE, Nemotron 3 Nano 4 B hybrid Mamba2–Transformer with reasoning, Gemma 4 26 B MoE with ~4 B active) to support the comparison at similar active compute in Fig 3 panel B and §5.2.

The roster also includes a **distillation pair** at the 9 B point and a **think/nothink toggle pair** on GLM 4.5.

3.5.1 Runtime and per-model identifiers

Table 2 — Model runtime details.

Per-model model identifier, runtime, protocols swept, and sample size. Quantisation for local models follows LM Studio's default at the time of the experiment (typically `GGUF Q4_K_M` for ≤14 B models and `GGUF Q4_0` for larger MoE checkpoints); the exact per-model quantisation is recorded in each `experiments/<model>/experiment.yaml` artefact alongside the LM Studio model identifier.

Model (paper)	Vendor model ID	Runtime	Protocols	N / arm	Reasoning
qwen3.5 0.8 B	`qwen3.5-0.8b`	LM Studio (local)	HTTP	200	—
qwen3.5 2 B	`qwen3.5-2b`	LM Studio (local)	HTTP	200	—
Gemma 4 E2B	`google/gemma-4-e2b`	LM Studio (local)	HTTP	200	—
Gemma 4 E4B	`google/gemma-4-e4b`	LM Studio (local)	HTTP	200	—
qwen3.5 4 B	`qwen3.5-4b`	LM Studio (local)	HTTP	200	—
Nemotron 3 Nano 4 B (hybrid Mamba2-Transformer)	`nvidia/nemotron-3-nano-4b`	LM Studio (local)	HTTP	200	think (default)
qwen3.5 9 B	`qwen/qwen3.5-9b`	LM Studio (local)	HTTP	200	—
qwen3.5 9 B distilled	`flipradio/qwen3.5-9b-distill` [1]	LM Studio (local)	HTTP	200	inherited (teacher)
Ministral 14 B	`mistralai/ministral-3-14b-reasoning`	LM Studio (local)	HTTP, MCP, A2A	200	think (default)
Gemma 4 26 B (MoE)	`gemma-4-26b-a4b-it`	LM Studio (local)	HTTP, MCP, A2A	200	—
GLM 4.5	`glm-4.5`	cloud (z.ai API)	HTTP, MCP, A2A × think/nothink	600	toggleable
gpt-oss-20b (excluded)	`openai/gpt-oss-20b`	LM Studio (local)	HTTP	50	think (default)

[1] Full LM Studio identifier: `flipradio_qwen_3.5_9b_deepseek_v4_flash_distillation` (community distillation of qwen3.5 9 B with DeepSeek-V4-Flash as teacher; abbreviated in the table for readability).

Sampling parameters (all cells): `temperature = 0.2`; per-run seed 2026 + 1000·k + j (§3.2), used by the chaos proxy and passed to the model API as its sampling seed; recorded per row.

Local hardware: LM Studio served from a Windows 11 / Intel Core i5-12400F (12 threads) / 128 GB RAM / NVIDIA RTX 3060 (~4 GB visible VRAM) workstation. Wall-time numbers in §4.7 should be read as comparisons within this study, not as absolute deployment estimates.

Cloud cell (GLM 4.5): served via z.ai API, `glm-4.5` endpoint, same `temperature = 0.2` and per-run seed. Reasoning toggled via the API's `thinking_mode` parameter.

3.6 Caveats and excluded model

Capacity floor (qwen3.5 0.8 B). This model stays in the roster (so the figures show the floor) but its `nopb` and `pb` arms both score 0 % at all rates—it cannot follow the task at chaos = 0 %. We treat it as the empirical capacity floor and discuss the implications in §5.

Structural rescue (qwen3.5 2 B). Its `nopb` arm scores 0 % at chaos = 0 % because the model cannot emit valid structured output without the playbook prompt. The +56.5 pp gap at chaos = 10 % therefore reflects a *structural rescue* (the playbook supplies missing format scaffolding) rather than purely *chaos rescue*. We retain the data point because the gap profile across rates is still informative, but flag the caveat throughout.

Excluded model (`gpt_oss_20b`) [30]. At chaos = 0.20 the model entered a reproducible degenerate decoding loop (hundreds of consecutive `?` tokens after five identical errors on the same endpoint; seed 2065, `pb` arm), a language-model-level failure qualitatively distinct from the trajectory failures of the active roster. The case is documented with its environment in `experiments/gpt_oss_20b/MODE_COLLAPSE_pb-0.2-39.md`; that run is recorded as a synthetic failure row, and the sweep was not extended beyond 0.20. Rates 0.00–0.20 (800 runs) are preserved as appendix evidence under `experiments/gpt_oss_20b/`; we do not analyse this model further:

Table 3 — Excluded / partial models.

These models appear in the dataset but are excluded from the main comparative figures (all comparative figures) because their coverage or failure mode is qualitatively different from the active roster. Listed here as appendix evidence.

Model	Total params	Active	Reason for exclusion
gpt_oss_20b	21B	3.6B	Reproducible degenerate decoding loop ('?' tokens) at rate 0.20 (§3.6); rates 0.00–0.20 preserved as qualitative evidence of high-chaos breakdown but not comparable with the rest of the roster (different N + truncated rates).

3.7 Metrics

We report:

- **Success rate (SR)**: fraction of runs that complete the four steps in order with non-error responses (the oracle of §3.1). CIs are Wilson 95 % [35].
- **Δ (pb − nopb)**: difference in success rate, in percentage points, between the two arms at a fixed rate. Reported CIs on Δ use the normal approximation for the difference of two proportions

(`diff_ci_95` in `scripts/audit_paper_v2.py`); this treats the arms as independent and is therefore conservative for our paired design, and it can be inaccurate near 0 % or 100 %, where score-based intervals are preferable [36].
- **Paired significance**: because run *i* uses the same seed and task instance in both arms, each cell yields 200 (or 600) matched pairs. We test $\Delta \neq 0$ with the exact McNemar test on the discordant pairs (runs that succeed in only one arm) and control the family-wise error rate with the Holm procedure over the 81 functioning-roster HTTP cells with $p > 0$ (`scripts/paired_analysis.py`). GLM 4.5 enters this family in think mode; its nothink HTTP cells give the same picture (no run succeeds only without the playbook, and every cell from 2 % upwards is significant even under a Bonferroni bound over 81 tests).
- **Tokens / run**: prompt + completion tokens per run, mean. The playbook prompt itself is amortised across the run, so the per-run number captures both the larger system prompt and any extra retry turns.
- **Wall time / run**: end-to-end seconds per run, mean. Includes any retry latency the playbook introduces.
- **LLM calls / run**: number of model invocations per run, mean. Useful as a proxy for retry behaviour.
- `steps_completed` and `failed_at` (stored per row, not aggregated in the headline figures): used in §5 to characterise *where* in the trajectory each arm fails.

Every number quoted in the text can be recomputed from the released CSVs. The figures (Figs 1–8) and the generated tables (Table 1, roster; Table 3, excluded model; the runs-summary table) are produced by `scripts/generate_paper_figures.py`. The inline tables of §4 and the numbers quoted in the text are checked against the CSVs by `scripts/check_paper_numbers.py`, and the paired tests by `scripts/paired_analysis.py`. Table 2 (runtime details) is transcribed from the released `experiment.yaml` files.

3.8 What we held constant

To make the cross-model comparison clean:

- Same seed per run index in both arms (seed $2026 + 1000 \cdot k + j$, §3.2), hence the same initial fault schedule.
- Same per-request sampling parameters (temperature 0.2). Decoding is therefore not deterministic: a retried call can return a different model action, and the retried tool call draws a new fault decision.
- Same task instance: every run of every cell uses one fixed task (product `SKU-1234`), so run-to-run variation comes only from the fault schedule and from model sampling; variance across task instances is not measured.
- Same prompt skeleton and identical playbook text per protocol; only the playbook ON/OFF flag differs between arms.
- Same trace plumbing—every run emits an identical schema row.

Variance between arms is therefore attributable to (a) the playbook prompt, (b) any retry behaviour the playbook elicits and (c) the resulting differences in the chaos schedule encountered (because retries re-roll the chaos die).

4 Results

This section reports the empirical findings, illustrated by eight figures (Figs 1–8; files `paper/figures/fig2_*` to `fig9_*`) and computed from the per-cell CSVs (`experiments/<model>/<alias>/<protocol>/rate_<R>.csv`). Every number quoted below can be recomputed from those CSVs with `scripts/check_paper_numbers.py`.

4.1 The playbook closes the chaos-induced gap

Fig 1—Hare and tortoise (`paper/figures/fig2_hare_tortoise.svg`). Per-model success rates at a single high-chaos operating point (HTTP, chaos = 20 %), plotted against mean run duration (log scale). Hollow markers are `nopb`, filled markers are `pb`, and each arrow is one model's playbook lift; the numbered side legend maps each number to a model, ordered by `pb` success rate (descending).

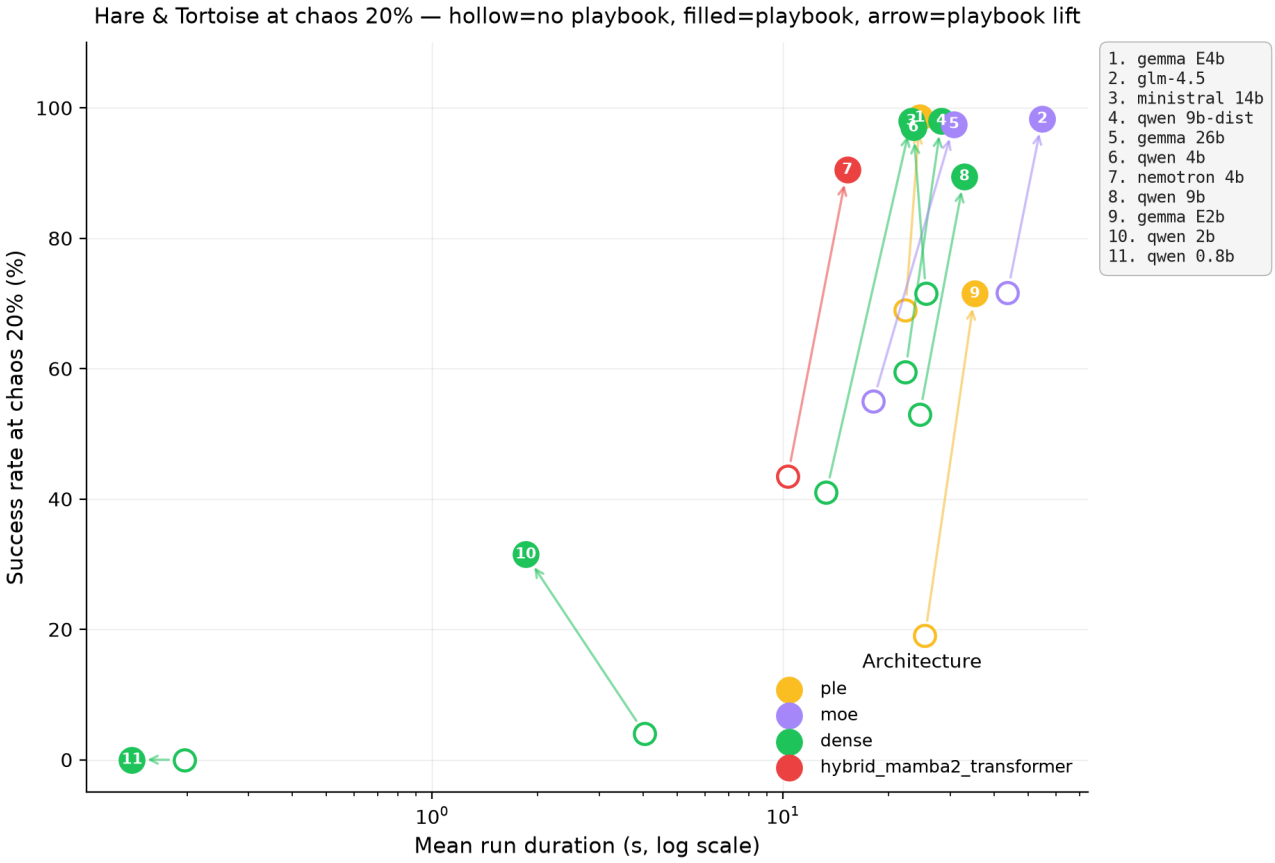


Table 4—Success rates at 20 % chaos (HTTP). SR in %, Δ in percentage points, computed from unrounded values.

Model	nopb SR	pb SR	Δ (pp)
qwen3.5 0.8 B	0.0	0.0	0.0
qwen3.5 2 B	4.0	31.5	+27.5
Gemma 4 E2B	19.0	71.5	+52.5
Gemma 4 E4B	69.0	98.5	+29.5
qwen3.5 4 B	71.5	97.0	+25.5

Model	nopb SR	pb SR	Δ (pp)
Nemotron Nano 4 B	43.5	90.5	+47.0
qwen3.5 9 B	53.0	89.5	+36.5
qwen3.5 9 B (distilled)	59.5	98.0	+38.5
Ministral 14 B	41.0	98.0	+57.0
Gemma 4 26 B (MoE)	55.0	97.5	+42.5
GLM 4.5 (think)	71.7	98.3	+26.7

Headline: every active model above the capacity floor benefits from the playbook, and at 20 % chaos the `pb` arm lies between **89 % and 99 %** for the upper-roster models (qwen3.5 9 B and above): the playbook *flattens* the success-rate distribution at the top, regardless of where the model started. No model has a negative Δ at any rate (under the transient faults we inject; see §5.5 xi).

The design is paired—run i uses the same seed and task instance in both arms—so the effect can be tested on matched pairs. Table 5 counts, for each functioning-roster model, the runs that succeed only with the playbook and only without it, and reports the exact McNemar test with Holm correction over the 81 functioning-roster HTTP cells with $p > 0$. The playbook effect is significant in 68 of the 81 cells; the 13 non-significant cells are all at failure rates of 1 %–3 %, where few calls fail and there is little to recover. At 10 % and 30 % chaos every model is significant (Holm-adjusted $p < 10^{-4}$), and runs that succeed only *without* the playbook are rare (0–9 of 200; 0–1 of 600 for GLM 4.5).

Table 5—Paired comparison (HTTP). Runs that succeed only with the playbook / only without it, out of $N = 200$ pairs (GLM 4.5: 600).

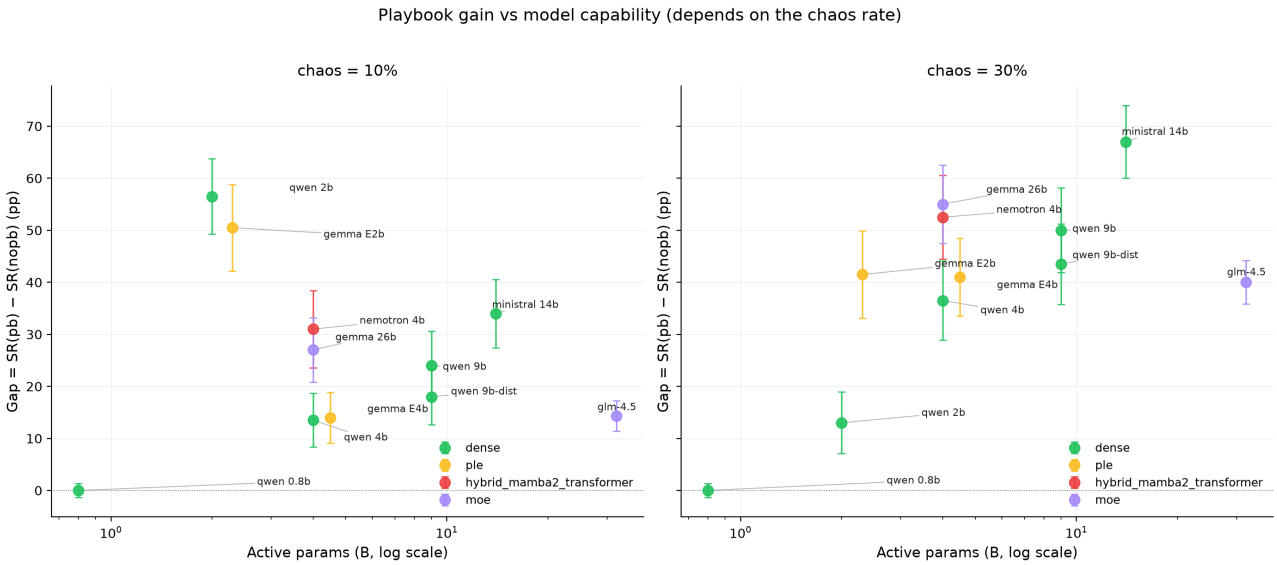
Model	10 % chaos	30 % chaos
Gemma 4 E2B	107 / 6	92 / 9
qwen3.5 4 B	29 / 2	76 / 3
Gemma 4 E4B	28 / 0	82 / 0
Nemotron Nano 4 B	71 / 9	109 / 4
Gemma 4 26 B (MoE)	54 / 0	110 / 0
qwen3.5 9 B	48 / 0	100 / 0
qwen3.5 9 B (distilled)	36 / 0	88 / 1
Ministral 14 B	68 / 0	134 / 0
GLM 4.5 (think)	87 / 1	240 / 0

qwen3.5 0.8 B (capacity floor) and qwen3.5 2 B (structural rescue) are excluded from this table and from the 81-cell family (§3.6, §5.3).

4.2 Where the playbook helps most

Fig 2—Δ vs active parameters at chaos = 10 % and chaos = 30 % (`paper/figures/fig3_hump.svg`). Two panels on a shared y-axis: x is model active parameters (log scale, B); y is Δ(pb − nopb). Architecture encoded by colour, error bars are 95 % CIs on the difference of

two proportions. The left panel shows a moderate chaos rate; the right panel shows the highest rate tested.



The relation between Δ and model size **depends on the chaos rate**. At chaos = 30 % the largest gain is at 14 B (Ministral, +67 pp); the ~4 B–9 B models gain +36.5 to +55 pp, the ~2 B PLE model +41.5 pp and the frontier model +40 pp (32 B active). At chaos = 10 % there is no such pattern: the largest gain is the smallest functioning model (Gemma 4 E2B, ~2.3 B effective, +50.5 pp), whose `nopb` baseline is already weak, and the ~4 B models range from +13.5 to +31 pp. A quadratic fit of Δ on log(active parameters) over the functioning roster is concave at 30 % (peak near 10 B) and convex at 10 %, and neither curvature is statistically significant at n = 9 (§5.5 viii). Values at 10 % chaos (the qwen3.5 0.8 B floor and the qwen3.5 2 B structural-rescue case are omitted):

Table 6—Δ(pb − nopb) at 10 % chaos (HTTP), functioning roster.

Active params (B)	Model	Δ at 10 % chaos
~2.3	Gemma 4 E2B	+50.5
4	qwen3.5 4 B	+13.5
~4.5	Gemma 4 E4B	+14.0
4	Nemotron Nano 4 B	+31.0
9	qwen3.5 9 B	+24.0
9	qwen3.5 9 B (distilled)	+18.0
14	Ministral 14 B	+34.0
~4 (active, MoE)	Gemma 4 26 B	+27.0
32 (active, MoE)	GLM 4.5 (think)	+14.3

Two observations.

(a) Saturation when `nopb` is already high. GLM 4.5 (`nopb` SR at chaos = 10 % is 85.2 %, `pb` SR is 99.5 %) extracts a +14.3 pp gain—comparable to the 4 B-class dense models, and smaller than every model in the 9 B–26 B band. The mechanism is not specifically *frontier* saturation: qwen 3.5 4 B (`nopb`

= 85.5 %, Δ = +13.5 pp) and Gemma 4 E4B (`nopb` = 86.0 %, Δ = +14.0 pp) sit at the same operating point within Wilson 95 % CIs at *N* = 200 / 600. What controls Δ at saturation is **proximity of the `nopb` baseline to the playbook ceiling**, not parameter count. The frontier model has a higher *floor at extreme chaos* (it does not break the way smaller models do—its `nopb` at chaos = 30 % is 57.0 %, comparable to qwen 4 B's 57.5 % within CIs), but at the canonical operating rates where `nopb` is already in the 80–90 % band the marginal value of the playbook is bounded by how much headroom remains. The same inverted-U pattern has been independently observed for *prompting strategies* on math reasoning [40]: a recent "Prompting Inversion" study finds a sophisticated prompting method peaks on gpt-4o and collapses to parity on gpt-5, via what the authors term a "Guardrail-to-Handcuff transition"—the constraint helpful for the mid-tier model becomes neutral or counterproductive for the frontier. Our observation at high chaos is consistent with this independent finding on a different axis.

(b) A mid-range maximum at high chaos only. At chaos = 30 % the largest gain is Ministral 14 B (+67 pp); at chaos = 10 % Gemma 4 E2B (+50.5 pp) leads because its `nopb` baseline is already weak (it gains partly from the structure the playbook prompt adds, a milder version of the qwen3.5 2 B structural-rescue case), and Ministral (+34 pp) is second. "Mid-range models gain the most" therefore describes the high-chaos regime, not the moderate one. See Fig 3 panel C.

At chaos = 30 % the gap profile is:

Table 7—Δ(pb − nopb) at 30 % chaos (HTTP), functioning roster.

Model	Δ at 30 % chaos
Gemma 4 E2B	+41.5
qwen3.5 4 B	+36.5
Gemma 4 E4B	+41.0
Nemotron Nano 4 B	+52.5
qwen3.5 9 B	+50.0
qwen3.5 9 B (distilled)	+43.5
Ministral 14 B	**+67.0**
Gemma 4 26 B (MoE)	+55.0
GLM 4.5 (think)	+40.0

The gap at chaos = 30 % is larger than at chaos = 10 % for every functioning model except Gemma 4 E2B (+50.5 → +41.5 pp): within the band where the model can follow the task at all, the playbook's value grows with environmental hostility.

4.3 The full sweep, by capability bucket

Fig 3—Cross-model SR vs chaos rate (`paper/figures/fig4_cross_model.svg`). A 2×2 grid splitting the active roster into four capability brackets. Colour encodes vendor (Alibaba blue, Google yellow, NVIDIA green, Mistral violet, Zhipu pink, the community-distilled model grey); within a panel, marker shape distinguishes models of the same vendor. Solid lines are `pb`, dashed lines are `nopb`.

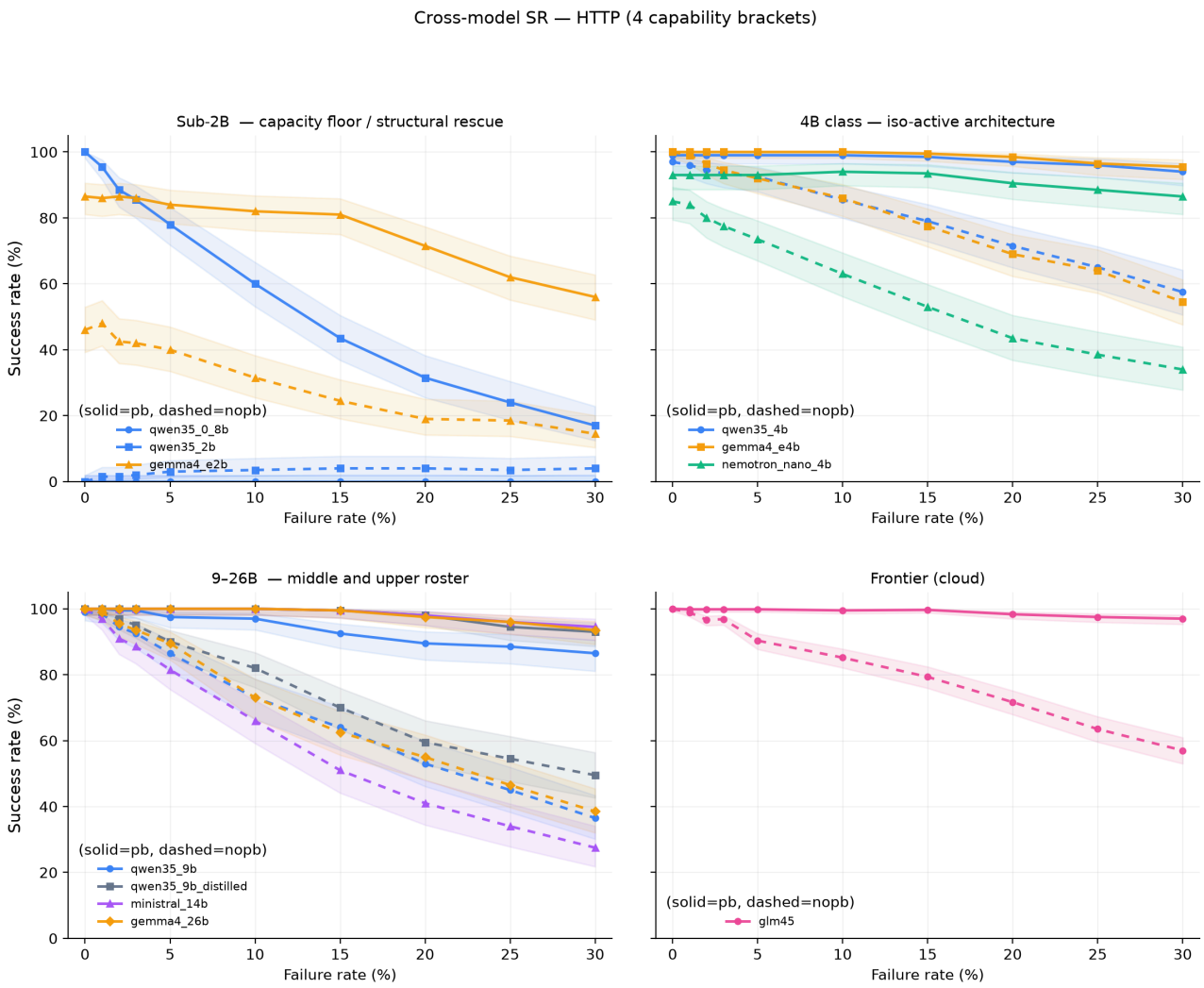


Table 8—Capability brackets of Fig 3.

Panel	Bucket	Models
A	Sub-2B—capacity floor / structural rescue	qwen3.5 0.8 B, qwen3.5 2 B, Gemma 4 E2B
B	~4 B class—similar active compute, different architectures	qwen3.5 4 B, Gemma 4 E4B, Nemotron Nano 4 B
C	9 B–26 B total—middle and upper roster	qwen3.5 9 B, qwen3.5 9 B distilled, Ministral 14 B, Gemma 4 26 B
D	Frontier (cloud)	GLM 4.5

Each panel plots success rate vs failure rate for both arms, with Wilson 95 % bands. Three takeaways:

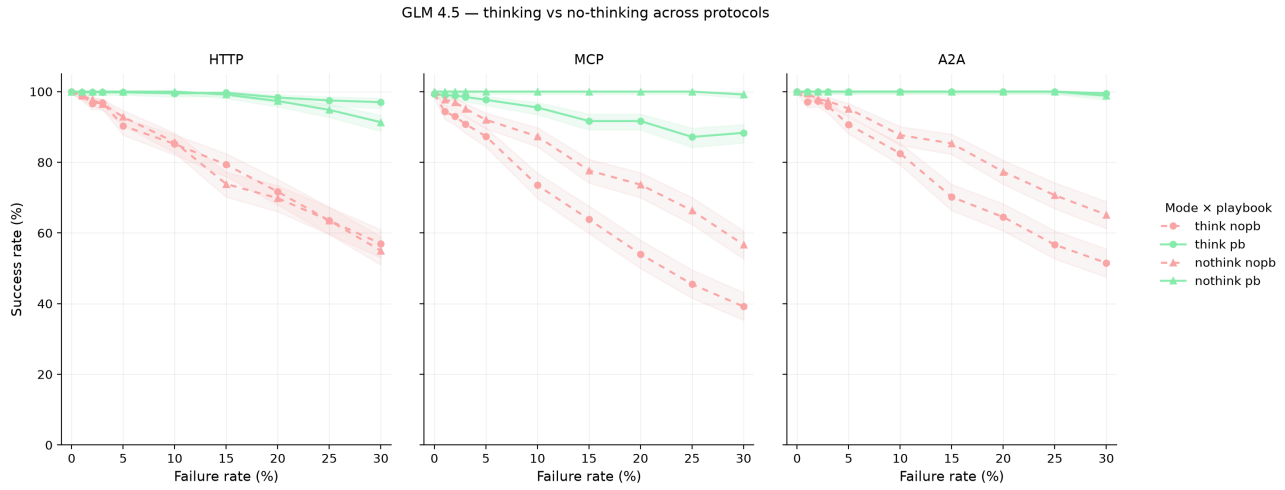
- **Panel A** (Sub-2B): qwen3.5 0.8 B is flat at zero in both arms (capacity floor). qwen3.5 2 B's `nopb` arm is at the floor at every rate (0 % to ~4 %); the `pb` arm starts at 100 % at chaos = 0 % and degrades to 17 % at chaos = 30 %. Gemma 4 E2B is the most informative member of this bucket—both arms function and the playbook gap widens with chaos, mirroring the larger models.
- **Panel B** (~4 B class): the two non-reasoning models (qwen3.5 4 B dense, Gemma 4 E4B PLE) stay within 3 pp of each other on the `nopb` arm at every rate; the reasoning hybrid Mamba2–Transformer model (Nemotron 3 Nano 4 B) sits 12–26 pp below them. The `pb` arm is essentially

saturated for qwen3.5 4 B and Gemma 4 E4B (≥ 94 % at chaos = 30 %) and slightly behind for Nemotron Nano 4 B (86.5 % at 30 %). Nemotron has the *lowest* `nopb` of the three at chaos = 0 % (85 %) and the *largest* `pb – nopb` gap at chaos = 30 % (+52.5 pp). We cannot disentangle two confounded effects in this panel: reasoning-by-default introducing self-induced drift (consistent with the §4.4 think/nothink evidence on GLM 4.5 under MCP/A2A) versus the hybrid Mamba2–Transformer architecture itself. Running Nemotron with its reasoning toggle off would separate them; we did not vary it in this campaign.

- **Panel C** (9 B–26 B): all four models reach `pb` ≥ 86 % at chaos = 30 %. The MoE model (Gemma 4 26 B, ~4 B active) behaves like the dense 9 B–14 B models on both arms. The community-distilled qwen3.5 9 B scores above the original on `nopb` at every chaos rate (gap +1 pp at chaos = 0 % widening to +13 pp at chaos = 30 %, individually significant only at 10 % and 30 %; §4.6), and on `pb` it saturates at 100 % up to chaos = 10 % and stays 6–8.5 pp above the original at every higher rate.
- **Panel D** (Frontier): GLM 4.5 (think mode, 32 B active of 355 B) maintains a `nopb` floor of 57.0 % at chaos = 30 %—among the highest in the HTTP roster, statistically indistinguishable from qwen3.5 4 B (57.5 %) within Wilson 95 % CIs (GLM at $N = 600$ has Wilson half-width ≈ 4.0 pp vs qwen3.5 4 B at $N = 200$ with ≈ 6.8 pp)—and reaches `pb` = 97.0 % there. The arms are visibly closer than for any other model in the roster, which is the saturation signature: the unguided model is already close to the playbook ceiling.

4.4 Thinking (GLM 4.5): no detectable effect on the unguided arm with flat tools, harmful on structured-error protocols

Fig 4—Think vs nothink across protocols (`paper/figures/fig5_thinking.svg`). Three panels, one per protocol (HTTP, MCP, A2A). Each panel plots GLM 4.5 for both arms. Colour and line style encode the arm (pastel green and solid = `pb`; pastel red and dashed = `nopb`), with marker shape distinguishing think (○) from nothink (△).



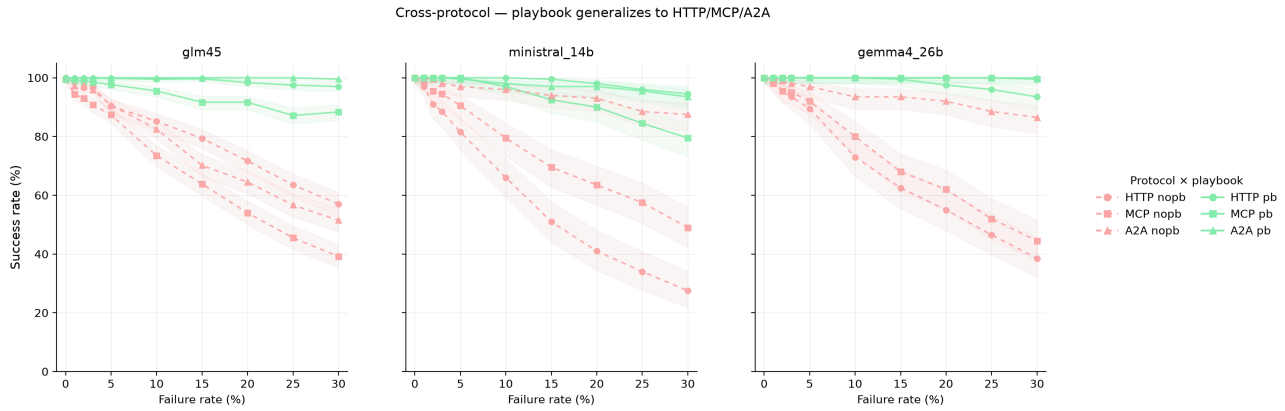
The effect of the think toggle is **asymmetric across protocols**. On **HTTP** the think and nothink `nopb` curves are within Wilson 95 % CIs at every chaos rate (maximum observed difference +5.5 pp at chaos = 15 %; at chaos = 30 %: think 57.0 % vs nothink 55.0 %, Δ = +2.0 pp ± 5.6)—explicit reasoning has no detectable effect on the unguided arm with flat HTTP tools. On **MCP** and **A2A**, however, think actively *hurts* the unguided arm well outside CIs: at chaos = 30 % MCP, `nopb` drops from 56.7 % (nothink) to 39.2 % (think); on A2A from 65.2 % to 51.5 %. We hypothesise that the typed error

semantics of MCP/A2A (`timeout`, `rate_limited`, `task_rejected`) invite over-deliberation—the model reasons about the error type instead of treating it as a recoverable signal—and the longer reasoning chains drift off-task more often than a single HTTP status code would. This hypothesis is consistent with the aggregate `nopb` SRs but not directly supported by trace-level inspection in this paper (§5.5 ix).

With the playbook the picture depends on the protocol. On A2A the playbook absorbs the would-be reasoning benefit (chaos = 30 % `pb`: think 99.5 vs nothink 98.8). On HTTP it does not fully: think keeps a 5.7 pp advantage (97.0 vs 91.3, outside Wilson 95 % CIs at N = 600), so with flat tools explicit reasoning still adds reliability on top of the playbook. The exception is **MCP under think**, where `pb` drops to 88.3 % at chaos = 30 % (vs 99.2 % under nothink): the same over-deliberation that hurts MCP `nopb` leaks through the playbook, attenuated but visible. Constructive observation for deployers: the playbook recovers most of the recovery benefit of explicit reasoning (fully on A2A, not fully on HTTP) at a fraction of the inference cost (see §4.7), and on structured-error protocols it is actively safer to *disable* explicit thinking unless a downstream task requires it.

4.5 The playbook transfers across protocols, except MCP under reasoning

Fig 5—Cross-protocol (`paper/figures/fig6_cross_protocol.svg`). The three models with full protocol coverage—Ministral 14 B (HTTP/MCP/A2A think), Gemma 4 26 B (HTTP/MCP/A2A) and GLM 4.5 (HTTP/MCP/A2A, think + nothink)—are plotted side-by-side. Markers encode protocol (○ HTTP, ▢ MCP, △ A2A); colour encodes arm (pastel green = pb, pastel red = nopb); line style separates pb (solid) from nopb (dashed).



Two observations:

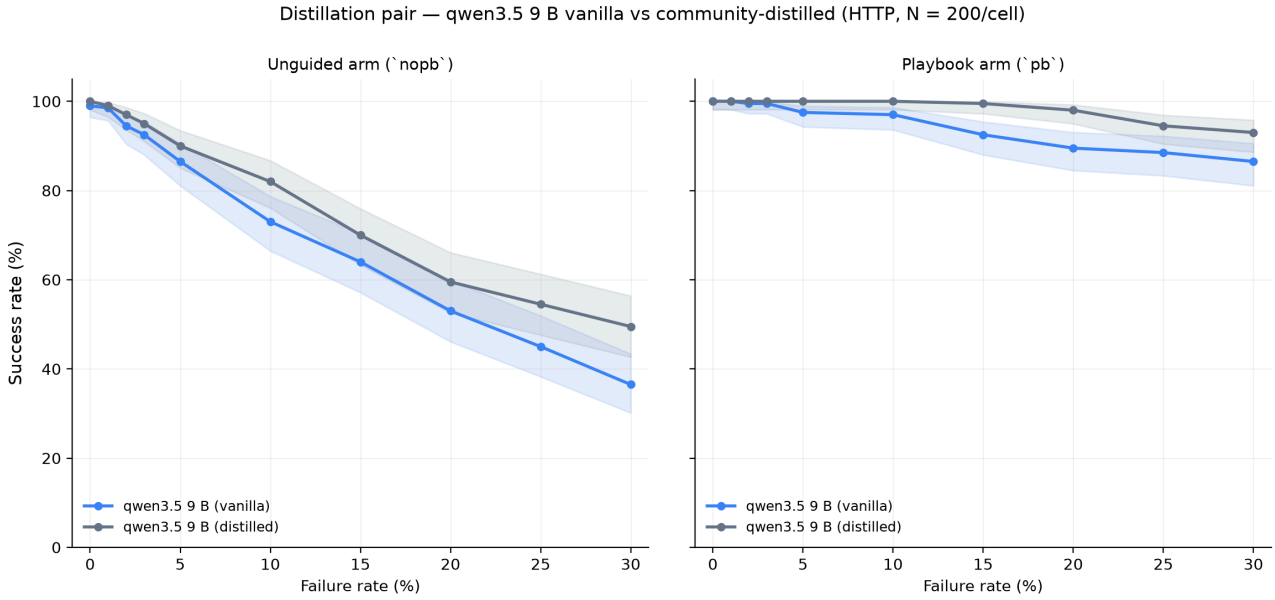
- The `pb` arm **mostly generalises across protocols**: at chaos = 30 % it lies between 88 % and 100 % for the three models, except Ministral 14 B on MCP (79.5 %). The same recovery instructions, adapted to each protocol's error vocabulary, produce comparable success on HTTP, MCP and A2A. The one consistent dip is MCP under think mode (Ministral 14 B: MCP `pb` 79.5 % vs HTTP 94.5 % / A2A 93.5 %; GLM 4.5 think: MCP `pb` 88.3 % vs HTTP 97.0 % / A2A 99.5 %), the same pattern surfaced in §4.4: explicit reasoning interferes with structured-error recovery even when the playbook is active.
- The `nopb` curve has more cross-protocol spread, and **the direction of the spread is model-dependent**. For **Ministral 14 B** and **Gemma 4 26 B**, A2A is the *most forgiving* environment (chaos = 30 % `nopb`: Ministral A2A 87.5 % vs HTTP 27.5 %, a +60 pp gap; Gemma 26 B A2A 86.5 % vs HTTP 38.5 %, +48 pp). For **GLM 4.5 in think mode** the pattern flips—HTTP is best (57.0

%), MCP worst (39.2 %), A2A in the middle (51.5 %). In **GLM 4.5 nothink** mode, A2A is best again (65.2 %), echoing the mid-roster pattern.

The constructive interpretation has two parts. First, **the playbook brings most protocol/model combinations up to a similar ceiling** (90–100 % at chaos = 30 %), with **MCP under reasoning the documented exception**—both Ministral 14 B (MCP `pb` = 79.5 %) and GLM 4.5 think (MCP `pb` = 88.3 %) sit visibly below their HTTP/A2A `pb`. We do *not* claim the playbook fully equalises across protocols; on this single recovered case the protocol-specific structural disadvantage of MCP under reasoning leaks through. Second, the protocol-specific *floor*—i.e. what the unguided model achieves—is shaped by how much **structural scaffolding** the protocol gives the model for free. In our A2A rig each step is phrased as a delegated task and chaos arrives as typed task errors (`task_rejected`, `timeout`), which may be easier to recover from than a bare HTTP status code; this is consistent with the structural help that mid-sized models (Ministral, Gemma 26 B) appear to absorb. The frontier model (GLM 4.5 think) does not benefit from this envelope—possibly because its own reasoning compensates—and on MCP/A2A the extra structure even costs `nopb` performance (§4.4). The playbook is the partial equaliser: it removes most of the protocol-specific structural advantage on the `pb` arm, while leaving the MCP-under-reasoning floor as a stubborn residual. We mark this as a hypothesis pending trace-level evidence (§5.5).

4.6 Does community distillation preserve chaos resilience?

Fig 6—Distillation pair (`paper/figures/fig8_distillation.svg`). Two panels comparing qwen3.5 9 B (original) and a community-distilled 9 B variant (DeepSeek-V4-Flash teacher) under identical HTTP chaos sweeps, $N = 200$ per cell.



On the `nopb` arm the distilled variant scores above the original at all ten chaos rates, with the gap widening as chaos increases; individually, most of these differences are within sampling noise:

Table 9—qwen3.5 9 B vanilla vs community-distilled, `nopb` arm (HTTP, $N = 200$ per cell).

Chaos rate	Vanilla nopb (%)	Distilled nopb (%)	Δ (pp)	95 % CI on Δ (pp)
0 %	99.0	100.0	+1.0	± 1.4 (not individually sig.)

Chaos rate	Vanilla nopb (%)	Distilled nopb (%)	Δ (pp)	95 % CI on Δ (pp)
5 %	86.5	90.0	+3.5	± 6.3 (not individually sig.)
10 %	73.0	82.0	**+9.0**	± 8.0 (sig.)
20 %	53.0	59.5	+6.5	± 9.7 (not individually sig.)
30 %	36.5	49.5	**+13.0**	± 9.6 (sig.)

Only the differences at 10 % and 30 % chaos are individually significant, so the evidence is the consistency of the direction across rates rather than any single difference. A sign test over the ten rates (10/10 in the same direction, two-sided $p = 0.002$) supports that consistency, under the assumption that the ten rates are independent comparisons; because both variants share the seed schedule, a paired analysis per rate would be the stricter test. We therefore read this as suggestive, not conclusive. On the `pb` arm the distilled variant sits 0–8.5 pp above the original at every rate; the playbook saturates both lines near 100 % up to chaos = 10 %.

This result inverts the direction of a preliminary internal observation made on an $N = 5$ distilled pilot (April 2026), which suggested the opposite—distilled appearing *more* fragile than vanilla. The pilot's apparent gap was within sampling noise; the $N = 200$ sweep settles it cleanly. Two empirical implications:

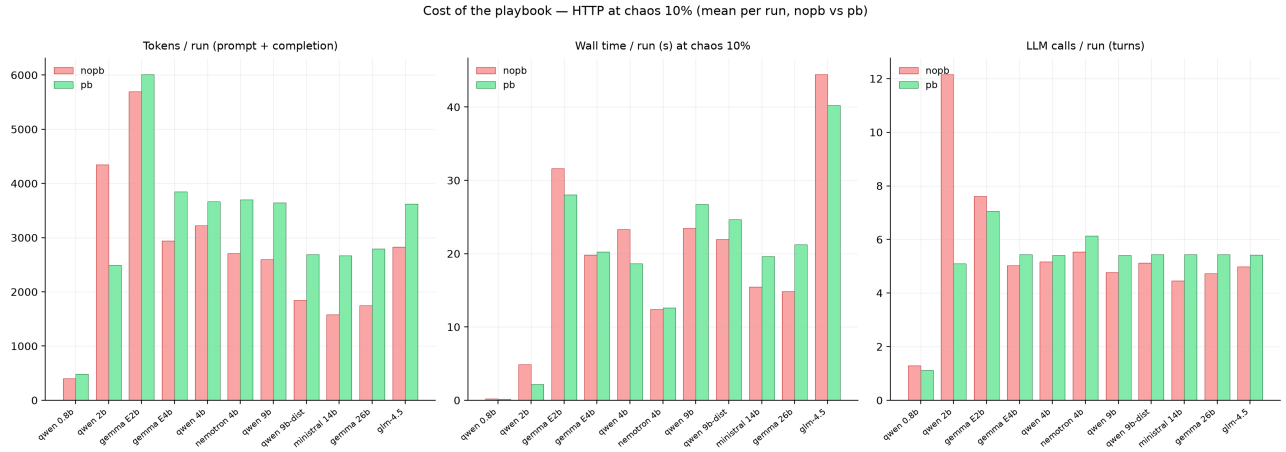
1. **Community distillation did not reduce chaos resilience at the 9 B point in our roster, and may modestly improve it.** The unguided distilled model scores above the unguided original at every chaos rate we tested, individually significant only at 10 % and 30 %. We do not interpret this as a general claim about distillation; it is one model pair, one task family. But it falsifies the conservative prior ("distillation can only lose") for this configuration.
2. **The playbook benefit does not depend on the distillation step.** Both variants saturate near 100 % at low chaos and stay above 86 % at chaos = 30 %; the playbook gain travels with the underlying capability bracket (§4.2) rather than with the training recipe.

Caveat: we deliberately do *not* claim a mechanism. The distilled variant emits zero `reasoning_tokens` per turn (the teacher's reasoning traces were not directly transferred), so we cannot attribute the gain to inherited deliberation pattern. A targeted ablation—pair vanilla vs distilled with matched compute budgets and matched system-prompt shapes—would be needed to identify the mechanism. We flag this as item 5 in §6.3 (the targeted distillation ablation).

Historical context: the closest prior art comes from image classification, where Papernot et al. [45] proposed defensive distillation against adversarial perturbations, a defence later shown to be ineffective by Carlini and Wagner [46]. Our observation concerns a different axis (benign, explicitly signalled tool failures in LLM agents) and does not rely on that line of work.

4.7 Cost is bounded and predictable

Fig 7—Cost of the playbook (`paper/figures/fig7_cost_of_playbook.svg`). Three panels at chaos = 10 % HTTP: tokens per run (prompt + completion, mean), wall time per run (s, mean), LLM calls per run (mean). For each panel, two bars per model—`nopb` (pastel red) and `pb` (pastel green). The figure shows every active model, with GLM 4.5 in think mode; Table 10 lists seven of them, with GLM 4.5 in nothink mode.



Numbers (per run, mean, at chaos = 10 % HTTP):

Table 10—Cost per run at 10 % chaos (HTTP), mean. Percentages are computed from unrounded values.

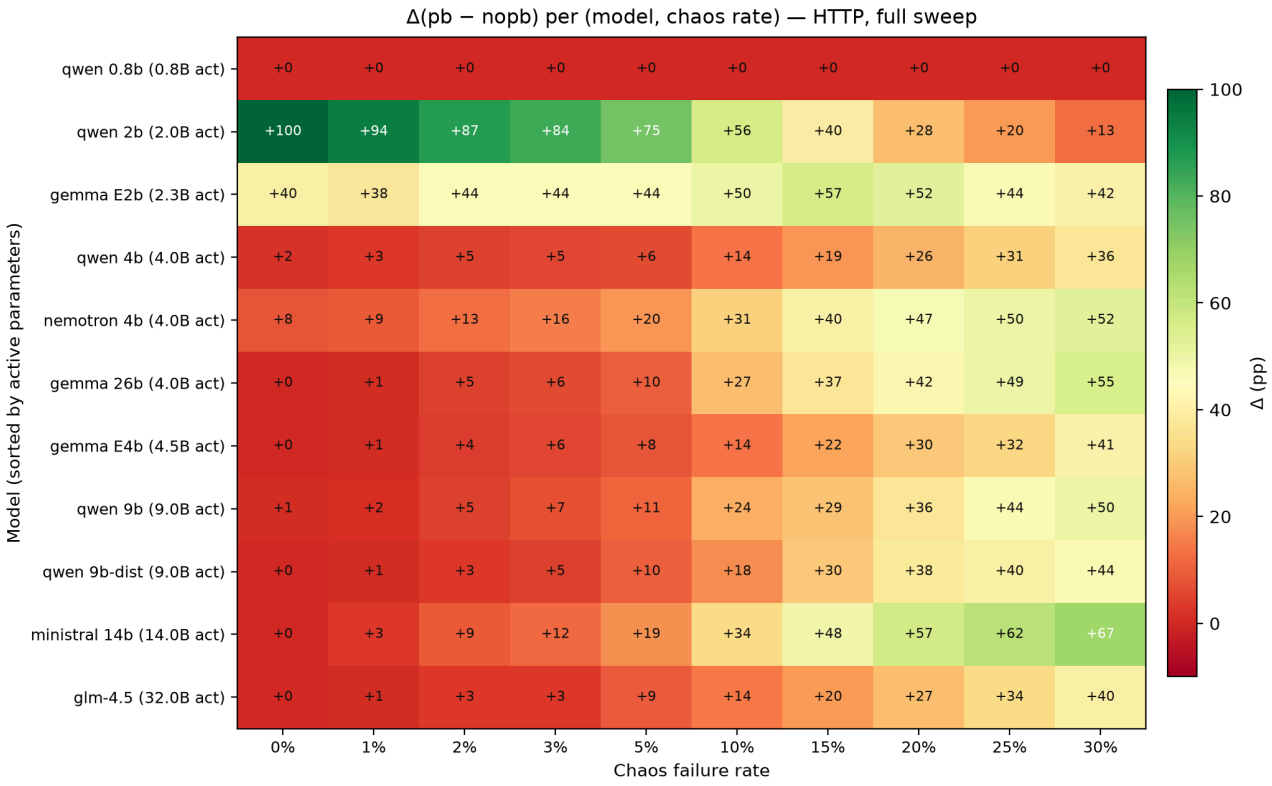
Model	nopb tokens	pb tokens	Δ tokens (%)	nopb calls	pb calls	nopb time (s)	pb time (s)
qwen3.5 4 B	3 224	3 663	+13.6 %	5.2	5.4	23.3	18.6
Gemma 4 E4B	2 941	3 846	+30.8 %	5.0	5.4	19.8	20.3
Nemotron Nano 4 B	2 709	3 702	+36.7 %	5.5	6.1	12.4	12.6
qwen3.5 9 B	2 597	3 644	+40.3 %	4.8	5.4	23.5	26.7
Ministral 14 B	1 581	2 664	+68.5 %	4.5	5.4	15.4	19.6
Gemma 4 26 B	1 750	2 791	+59.5 %	4.7	5.4	14.8	21.2
GLM 4.5 (nothink)	1 680	2 472	+47.1 %	5.0	5.4	14.5	13.4

- **Tokens**: the playbook adds 13 %–69 % (mean ≈ 42 %). The *absolute* cost is the playbook prompt itself (a few hundred tokens) plus the extra retry turns it elicits. The largest increase is Ministral 14 B (+68.5 %), whose retry chains include reasoning tokens; Nemotron Nano 4 B, the other reasoning-by-default model, sits mid-range (+36.7 %), so reasoning alone does not explain the spread.
- **LLM calls**: the playbook adds **+0.57 calls per run on average** (range +0.25 to +0.98 calls / run; mean across the seven cells in the table above). This is the empirical signature of the retry loop the playbook authorises.
- **Wall time**: model-dependent. Two cells run *faster* with the playbook (qwen3.5 4 B: −20 %; GLM 4.5 nothink: −7 %), two are essentially unchanged (Gemma 4 E4B, Nemotron Nano 4 B), and three run moderately slower (qwen3.5 9 B: +14 %; Ministral 14 B: +27 %; Gemma 4 26 B: +43 %). Faster runs come from the playbook abandoning long chaos-induced hallucination loops earlier; slower runs come from the extra retry latency dominating. GLM 4.5 is served through a cloud API (§3.5.1), so its wall times include network latency and provider-side scheduling and are comparable with the local timings only in sign.

The combined picture: a small fixed-shape token tax (~ +42 % per run on average, bounded above by ~ +69 % for the most reasoning-intensive models in the roster) buys the reliability gains documented in Figs 1–3. There is no free lunch but the lunch is cheap.

4.8 The full corpus at a glance

Fig 8—Δ(pb − nopb) heatmap across (model, chaos rate) (`paper/figures/fig9_heatmap.svg`). Rows are models sorted by active parameters (low → high); columns are the ten chaos rates (0 % → 30 %). Each cell is the per-rate Δ; the colour scale runs from red (Δ ≤ 0) through yellow (Δ ≈ 30 pp) to green (Δ ≥ 60 pp).



Four patterns are visible without reading any single curve:

1. **The capacity floor (qwen3.5 0.8 B, top row) is red across the entire grid**: the playbook cannot rescue what cannot follow the task.
2. **The structural-rescue band (qwen3.5 2 B, second row) is bright green at low chaos and fades to yellow at high chaos**: the playbook adds *structural* scaffolding that helps overwhelmingly at chaos = 0 % and progressively less as the underlying task becomes too hard for the 2 B model.
3. **The mid-roster (Gemma E4B / qwen 4 B / Nemotron / Gemma 26 B / qwen 9 B / Ministral) green band intensifies as chaos rises**: the playbook gain scales with environmental hostility for models that can already follow the task, with Ministral the brightest column at chaos = 30 %.
4. **The frontier row (GLM 4.5, bottom) is red-yellow at low chaos and yellow at high chaos**: the unguided model already approaches the playbook ceiling so the marginal value of the playbook is bounded.

4.9 Summary of the empirical map

Combining the eight figures and the paired tests:

1. **Across all functioning models the playbook helps and, under the transient faults we inject, never harms**, with a median Δ of **+24 pp** at chaos = 10 % (bootstrap 95 % CI over models: [14.0, 34.0]) and **+43.5 pp** at chaos = 30 % (bootstrap 95 % CI: [40.0, 55.0]) over the functioning roster (n = 9; the qwen3.5 0.8 B floor and the qwen3.5 2 B structural-rescue case are excluded, §5.3). In paired exact tests with Holm correction the effect is significant in every functioning-roster HTTP cell at failure rates of 5 % and above (Table 5).
2. **The marginal value of the playbook depends on capability and on the chaos rate.** At 30 % chaos it is largest in the mid-range (maximum at 14 B) and smaller for the frontier model, whose unguided baseline is already close to the playbook's ceiling; at 10 % chaos it is not monotonic in size. This is an observed pattern, not a formally tested law (§5.5 viii).
3. **The playbook generalises across the three interface styles** (HTTP/MCP/A2A, on the three models with full coverage) and across the one available reasoning toggle (GLM 4.5 think/nothink), with **MCP under reasoning** as the one documented exception, where the `pb` arm sits about 9–15 pp below its HTTP/A2A counterparts at 30 % chaos (§4.5).
4. **Capacity floor**: a 0.8 B model cannot use the playbook because it cannot follow the task. A 2 B model gains a +100 pp Δ at chaos = 0 % because the playbook supplies missing structural scaffolding.
5. **The cost of the playbook is bounded** (~ +42 % tokens on average; +0.57 extra LLM calls on average; wall time model-dependent—see §4.7).

These findings feed §5 (caveats and threats to validity) and §6 (what we recommend doing next).

5 Discussion

5.1 Why the gain depends on capability

At high chaos, Δ(pb − nopb) is largest for mid-range models and smaller both at the capacity floor and for the frontier model (§4.2). This observed pattern is consistent with two intuitions that meet in the middle.

(a) At the low end, the model may lack the capacity to *use* the playbook. qwen3.5 0.8 B is the demonstration: it cannot follow the four-step task at chaos = 0 %, and the playbook is just more text it does not act on. qwen3.5 2 B is a softer version of the same story—it cannot emit valid structured output without the playbook providing format scaffolding, so the +100 pp gap at chaos = 0 % is overwhelmingly *structural rescue* rather than chaos rescue (§4.1, §5.3). These two points define the **capacity floor** of the instrument.

(b) At the frontier, the model partly *invents* the playbook. GLM 4.5 (32 B active of 355 B total) reaches `nopb` SR = 85.2 % at chaos = 10 % and 57.0 % at chaos = 30 %—among the highest `nopb` arms in the HTTP roster, statistically indistinguishable from qwen3.5 4 B and Gemma 4 E4B within Wilson 95 % CIs at several rates (GLM at N = 600 vs the local roster at N = 200). Adding the playbook still helps (+14.3 pp at 10 %, +40.0 pp at 30 %), but the unguided model is already doing some of the

recovery the playbook codifies. The marginal value of the playbook shrinks because the baseline is close to the playbook ceiling.

Between these two ends lies the regime where a model has enough capability to *follow* recovery instructions but not enough to *invent* them on its own; in our roster, at 30 % chaos, this is where the playbook gains the most. The boundaries of that band are not sharp: at 10 % chaos the ~2 B PLE model gains more than any other, and the fitted curvature is not significant at n = 9 (§5.5 viii).

The practical implication is that the decision to add a playbook and the choice of model should be made together: below the capacity floor the playbook cannot rescue the agent, and near the frontier it still helps but less, because the unguided model already recovers on its own.

5.2 Architecture, not parameter count

In the ~4 B class (qwen3.5 4 B dense, Gemma 4 E4B PLE, Nemotron 3 Nano 4 B hybrid Mamba2–Transformer with reasoning), the two **non-reasoning** models stay within 3 pp of each other on `nopb` at every rate, and the `pb` arms of all three stay within ~9 pp. The reasoning hybrid model (Nemotron) drives the spread: it sits 12–26 pp below the non-reasoning pair on `nopb` but joins them on `pb`. The MoE model (Gemma 4 26 B, ~4 B active) behaves like the dense 9 B–14 B models; the roster has no dense 26 B model to compare it with. Taken together, the `pb` arm is similar across architectures at similar active compute, while the `nopb` floor is additionally modulated by *some* combination of reasoning-by-default and the hybrid architecture, which the Nemotron data alone cannot separate (§4.3 panel B).

A speculative observation: the reasoning model in the 4 B class (Nemotron Nano 4 B) has the lowest `nopb` baseline at chaos = 0 % and the largest `pb` − `nopb` gap at high chaos. We hypothesise (without definitive evidence in the current dataset) that internal reasoning introduces its own *self-induced* failure mode—long chains of thought that eventually drift off-task—and that the playbook absorbs both chaos-induced and reasoning-induced failures. The evidence is **protocol-specific**: the effect is visible in the Nemotron 4 B class (HTTP), and well outside CIs on **GLM 4.5 under MCP and A2A** (−17.5 pp on MCP, −13.7 pp on A2A at chaos = 30 % for think vs nothink, well outside Wilson 95 % CIs on the difference). On **GLM 4.5 under HTTP** the unguided think/nothink difference is within sampling noise at every rate (§4.4)—i.e. reasoning is approximately neutral on flat tools, harmful on structured-error protocols. We mark the broader "reasoning trades baseline reliability" reading as a hypothesis pending more reasoning-toggle pairs (§5.5 vi) and trace-level inspection (§5.5 ix).

5.3 Two caveats that must travel with the data

Capacity floor (qwen3.5 0.8 B). The model is in the comparative figures so the floor is visible, but the row should be read as "playbook cannot rescue what cannot follow instructions". It is a useful empirical anchor (we now know where the floor is) and should not be cited as evidence that the playbook ever harms a model.

Structural rescue (qwen3.5 2 B). The +100 pp Δ at chaos = 0 % is the playbook fixing a *format* problem (the model cannot emit valid JSON without explicit guidance) rather than a *chaos* problem. A reader who quotes the qwen3.5 2 B numbers without this caveat will misread the rest of the paper. We retain the row because the *shape* of the gap profile is informative: Δ decreases monotonically from +100 pp at chaos = 0 % through +56.5 pp at chaos = 10 % to +13 pp at chaos = 30 %, showing the playbook's chaos-rescue contribution shrinks as the absolute task becomes too hard for the underlying model.

In future work we will add a "structured-output baseline" arm—same model, same task, same prompt skeleton, plus an explicit JSON schema—to disentangle structural rescue from chaos rescue numerically.

5.4 Excluded: the mode-collapse model

We exclude `gpt_oss_20b` from comparative analysis because at chaos = 0.20 it entered a reproducible degenerate decoding loop—a *language-model-level* failure qualitatively distinct from the trajectory failures of the active roster, elicited here by benign, explicitly signalled errors at the tool boundary (§3.6). Rates 0.00–0.20 are preserved as appendix evidence (`experiments/gpt_oss_20b/`, with the reproducibility record of the failing run); the finding is flagged for follow-up but not analysed further here.

5.5 Threats to validity

(i) Single task family. The Petstore-style four-step workflow is representative of the structured-tool-use class but not of all agent tasks. A task with longer reasoning chains, ambiguous tool semantics or open-ended state would stress the playbook differently. The cross-protocol generalisation (Fig 5) provides a partial defence—the playbook works the same across HTTP, MCP and A2A *for this task*—but cross-task generalisation is future work.

(ii) Chaos schedule is uniform random. Our chaos rate is a marginal probability over independent calls. Real production failures are bursty, correlated and sometimes systemic (one downstream service is degraded for the next 30 s). A burst-mode chaos rig would likely change the shape of the curves; we expect the playbook to help *more*, not less, but this should be measured.

(iii) Sample sizes by cell. All local-roster cells are $N = 200$ per arm (qwen3.5 family, Gemma 4 family including the 26 B MoE, Nemotron Nano 4 B, Ministral 14 B); the cloud frontier model (GLM 4.5) is $N = 600$ per arm so that think × nothink × HTTP/MCP/A2A is well powered. The playbook effect is significant in paired exact tests with Holm correction in every functioning-roster HTTP cell at failure rates of 5 % and above (§4.1, Table 5); several secondary comparisons, such as the distillation pair at individual rates (§4.6) or GLM 4.5 vs qwen3.5 4 B, are within sampling noise and are reported as such. The released CSVs include the `seed` column for exact replication of the fault schedule.

(iv) One playbook per protocol. We did not sweep playbook variants. The HTTP playbook is 69 words; we have not measured how much of the gap is attributable to the playbook *content* versus the act of giving the model *any* recovery instruction. A control arm with a deliberately bad playbook would tell us. We anticipated negative results would have to be reported here; positive—i.e. the content matters—would be a stronger contribution.

(v) Local runtime variance. Most models in our roster ran on LM Studio (localhost). Throughput varies between machines and between runtime versions, and the cloud-served GLM 4.5 adds network latency and provider-side scheduling; wall-time numbers in §4.7 should be read as relative comparisons, not absolute deployment estimates. The token and call counts are runtime-independent.

(vi) Vendor reasoning toggle is single-model. Only GLM 4.5 in the roster exposes a clean think/nothink toggle. Generalising the §4.4 findings beyond GLM 4.5 requires more reasoning-toggle pairs.

(vii) One seed schedule. Most cells use seeds 2026 + *j*, so they share the same 200 (or 600) seeds. Cross-arm comparisons within a cell are therefore well controlled, but our error bars capture *binomial* sampling variance only, not variance due to the particular seed set. A partial check is available without new runs: in the GLM 4.5 MCP/A2A cells, part of the runs were collected in launches that gave them other seed offsets (3026 + *j*, 5026 + *j*, 7026 + *j*; §3.2). At chaos = 10 % and 30 % the success rates of the batches mostly agree closely (e.g. A2A nothink at 30 %: 65 % with seed base 2026, $N = 100$, and 65 % with base 3026, $N = 500$; MCP think at 10 %: 72 %, 74 % and 71 % across three bases); the largest difference is A2A think at 30 % `nopb` (61.0 % vs 49.6 %, $N = 100$ vs 500), about 2.1 binomial standard errors apart (two-sided $p \approx 0.03$)—borderline, and the main motivation for the seed-replication study proposed in §6.3. A dedicated replication with 3–5 alternative seed bases on a subset of the local roster would close this gap.

(viii) Multiple comparisons and the shape of the capability effect. For the main effect we control the family-wise error rate with the Holm procedure over the 81 functioning-roster HTTP cells (§3.7); other comparisons in §4 are descriptive and are not corrected. The dependence of Δ on model size is not formally established: a quadratic regression of Δ on log(active parameters) over the n = 9 functioning roster gives a negative (concave) coefficient at chaos = 30 % (peak near 10 B active parameters, $t \approx -1.1$) and a positive (convex) coefficient at chaos = 10 % ($t \approx +0.9$); neither is significant at n = 9. The size pattern, frontier saturation and the MCP-under-reasoning dip are therefore observations to be confirmed with more models, not tested findings. The bootstrap interval on the median Δ ([40.0, 55.0] at 30 %) quantifies the *size* of the playbook effect across models, not the shape of its dependence on size.

(ix) Mechanistic claims pending trace-level evidence. The hypotheses in §4.4 (over-deliberation drives MCP/A2A drift) and §4.5 (structural scaffolding from A2A's typed errors helps mid-sized models) are consistent with the aggregate `nopb` SRs we report but are *not* directly supported by trace-level inspection in this paper. Both hypotheses are falsifiable from the released CSVs (which include `reasoning_tokens` per turn and per-step `failed_at` markers) and we flag the trace-level study as immediate follow-up rather than as a settled finding here.

(x) Runtime under-specification. Local cells were served by LM Studio; specific runtime / driver / quantisation choices per model are recorded in the released `experiments/<model>/experiment.yaml` artefacts rather than transcribed into this paper. Headline SR differences are large relative to plausible runtime-driven variance, but wall-time numbers in §4.7 should be read as ordinal comparisons within this study, not as absolute deployment estimates.

(xi) A retry-propensity reading, and missing baselines. Because every injected fault is transient and the playbook tells the model to retry, much of the measured effect can be read as the playbook *inducing retries*. A simple model makes this concrete: if each of the four steps fails independently with probability p and is never retried, the expected success rate is $(1 - p)^4$; with up to three retries per step it is $(1 - p^4)^4$. Ministral 14 B without the playbook makes no retries at all and its success rate matches the first expression (66.0 % vs 65.6 % at $p = 0.10$; 41.0 % vs 41.0 % at 0.20; 27.5 % vs 24.0 % at 0.30); with the playbook it retries 1.3 times per run at $p = 0.20$ and reaches 98.0 %, close to the second expression (99.4 %). Across the functioning roster at $p = 0.30$, mean retries per run rise from 0.0–0.8 without the playbook to 1.7–2.4 with it. This reading does not diminish the practical result—several current models do not retry transient failures on their own, and a 69-word instruction changes that—but it bounds its interpretation. Three baselines that would sharpen it were not run in this campaign: (a) an LLM-free deterministic retry policy (the released `agents/deterministic.py`), which sets the

reliability ceiling a conventional client achieves without any model; (b) a minimal one-line instruction ("retry a failed call") and a length-matched neutral text, to separate the content of the playbook from the presence of extra instructions; and (c) a mix of transient and permanent faults, under which an "always retry" playbook could waste the turn budget—so the absence of harm reported here holds for transient faults only. Concurrent work reports the same lever in a different setting: in AgentCheck [44], a retry mitigation raised one agent's success under timeouts from 30 % to 100 % (a single agent on an MCP fault-injection workbench, not a cross-model comparison).

(xii) Emulated interfaces and an HTTP-only circuit breaker. All three rigs run in-process against mock tools: the HTTP rig answers from a mock Petstore and the MCP and A2A rigs emulate each protocol's message shape and error vocabulary, without network transport or official SDKs. The results therefore speak to how models react to each interface's error semantics, not to the behaviour of real protocol stacks. In addition, the HTTP rig wraps the proxy in a circuit breaker (§3.2) that blocks calls for 30 s after three consecutive failures; the MCP and A2A rigs have none. From the traces, among the functioning-roster HTTP cells and GLM 4.5 (think and nothink), a run sees at least one blocked call in 0.0 % (`nopb`) and 0.1 % (`pb`) of runs at 10 % chaos, 0.5 % and 3.5 % at 20 %, and 1.4 % and 8.9 % at 30 %; such runs succeed about 34 %–40 % of the time with the playbook. Because the breaker mostly penalises the arm that retries, it biases the HTTP playbook effect downwards (the reported HTTP Δ is conservative), and it contributes to the lower HTTP `pb` success relative to MCP and A2A in §4.5. Its wall-clock cooldown also interacts with model speed: faster models issue more calls while the breaker is open. Finally, the turn budget differs by protocol (15 turns for HTTP, 12 for MCP and A2A, §3.1). In the cross-protocol cells at 30 % chaos (Ministral 14 B, Gemma 4 26 B, GLM 4.5), budget exhaustion is rare: no `nopb` run exhausts it except 0.7 % in GLM 4.5 A2A nothink, and `pb` runs exhaust it in at most 1.2 % of MCP and A2A runs but in 2.0 %–8.7 % of HTTP runs, consistent with retries blocked by the circuit breaker. The tighter MCP/A2A budget therefore does not explain the cross-protocol differences of §4.5.

5.6 What does this mean for an SRE?

Three concrete recommendations for someone deploying an LLM agent today:

1. **Add a recovery playbook, and check that it matches your failure modes.** The cost is bounded (about +42 % tokens and +0.6 LLM calls per run) and the benefit is large (median +24 pp at chaos = 10 %, +43.5 pp at chaos = 30 %) for transient faults; a playbook that says "always retry" is only safe when faults are in fact transient (§5.5 xi).
2. **Choose the model and the playbook together.** In our roster, the models from qwen3.5 4 B to Gemma 4 26 B reach 86.5 %–95.5 % success at 30 % chaos with a playbook, close to the frontier model with a playbook (97 %) at a fraction of its size; a model below the capacity floor cannot be rescued by the playbook. Clean-benchmark rankings do not predict this.
3. **Run a chaos sweep before production**, not after the first incident. A sweep of one model at a few failure rates with a reduced N takes a few hours on a single workstation (the full ten-rate HTTP sweep of one local model at $N = 200$ per arm took between half a day and a day and a half in our setup); the released `chaos-engine` rig is open source. The marginal value of even one chaos sweep is large because failure modes (especially mode collapse, see §5.4) are easy to miss on clean benchmarks and hard to debug in production.

5.7 Compliance evidence (EU AI Act)

The chaos rig and the released CSVs are structured so that they could support the robustness evidence a provider of a high-risk AI system needs under the EU AI Act: the risk-management system of Article 9 and, in particular, the resilience to "errors, faults or inconsistencies" arising from interaction with other systems required by Article 15(4). Each row records the seed, the chaos rate, the protocol, the agent identity and the outcome, which gives an auditable trail from an agent configuration to the chaos sweep that exercised it. We do not claim that this discharges Article 15 by itself; we propose it as a candidate template for the measurement methodology the regulation calls for (Art. 15(2)) and which the LLM-agent literature does not yet routinely produce.

6 Conclusions

6.1 What we showed

We treated chaos engineering as a scientific instrument and used it to map the reliability surface of LLM agents across **eleven open and frontier models** (0.8 B to 355 B), four architecture types (dense, Per-Layer-Embedding, hybrid Mamba2–Transformer, MoE), three emulated agent–tool interface styles (HTTP, MCP, A2A), a think/nothink toggle and ten failure rates from 0 % to 30 %, totalling **128,000 active agent runs**. For each model we ran a paired A/B comparison between an unguided agent and the same agent under a small structured **recovery playbook**.

Three findings stand:

1. **The playbook helps and, under the transient faults we inject, never harms.** Median Δ(pb − nopb) over the **functioning roster** (n = 9, excluding the capacity-floor and structural-rescue cases) is +24 pp at chaos = 10 % and +43.5 pp at chaos = 30 %, significant in paired exact tests in every functioning-roster HTTP cell at failure rates of 5 % and above; on HTTP, the `pb` arm of the models from qwen3.5 4 B upwards stays between 86 % and 100 % at every chaos rate we tested.
2. **The benefit depends on capability and on the chaos rate.** A 0.8 B model is at the capacity floor; at 30 % chaos the mid-range models gain the most and the frontier model gains less, as its unguided baseline approaches the playbook's ceiling; at 10 % chaos the gain is not monotonic in size. This pattern is observed, not formally tested.
3. **The cost is bounded and predictable**: about +42 % tokens on average and less than one extra LLM call per run (≈ +0.57), with model-dependent wall time (−20 % to +43 %). There is no model in the roster where the playbook lowers the success rate under the transient faults we inject.

We also documented two qwen caveats (a capacity floor at qwen3.5 0.8 B and a structural-rescue artefact at qwen3.5 2 B) and showed that the playbook generalises across HTTP / MCP / A2A on the three models with full coverage, except MCP under reasoning.

6.2 What we did not show

To set expectations:

- We did not measure *task generalisation* beyond the four-step structured-tool-use workflow. Other task families (long-horizon planning, retrieval-heavy QA, tool composition under ambiguity) may exhibit different shapes.
- We did not vary the **content** of the playbook. We do not yet know how much of the gap is attributable to the specific recovery rules vs the act of providing any recovery scaffolding.
- We did not test **bursty** or **correlated** chaos. The rig is uniform-random per call.
- We did not compare the playbook with an **LLM-free deterministic retry policy** or with a **minimal retry instruction**, and we did not mix **permanent** faults with transient ones (§5.5 xi).
- We did not inject **silently corrupted payloads** (dropped fields, swapped types, truncated JSON). Every fault in this campaign was explicitly signalled to the agent; robustness to quiet corruption—an axis the engine's payload-corruption library supports—is left unexercised here.
- We did not run **multi-agent collusion** or **adversarial-environment** scenarios, which sit on a separate axis from the operational chaos studied here.

6.3 What we recommend doing next

The experimental campaign behind this paper is closed: ten local-roster cells are at $N = 200$ and the frontier cell (GLM 4.5) at $N = 600$, giving Wilson 95 % CIs in the figures we publish. The directions below are *follow-up work*, framed as separate studies rather than open tasks of this paper; we list them in rough order of value-per-effort.

1. **A seed-replication study**: repeat a subset of the local roster at two or three failure rates with 3–5 alternative seed bases, to measure seed-set variance directly (§5.5 vii).
2. **A "schema-only" control arm** to disentangle structural rescue from chaos rescue at the small-model end (qwen3.5 2 B and below). Same model, same task, same prompt skeleton, plus an explicit JSON schema. Closes the §5.3 caveat numerically.
3. **A "bad playbook" control arm** to measure how much of the gap is content-driven. The hypothesis is that a deliberately misleading playbook should hurt and a content-free recovery template should help less than the real one. If neither holds, the contribution narrative shifts toward "any recovery scaffolding works".
4. **More reasoning-toggle pairs.** GLM 4.5 is a single data point. Running Nemotron 3 Nano 4 B with its reasoning toggle off, and adding further think/nothink pairs, would let us replicate Fig 4 across families and separate reasoning from architecture in the ~4 B class.
5. **A targeted distillation ablation.** The community-distilled qwen3.5 9 B scores above the original on `nopb` at every chaos rate in our sweep, individually significant only at 10 % and 30 % (§4.6), but the distilled variant emits zero `reasoning_tokens` per turn, so we cannot attribute the gain to inherited deliberation. A controlled pair with matched compute budgets and matched system-prompt shapes would identify the mechanism. Extending the small-model cells (0.8 B / 2 B) to MCP and A2A would also test whether the structural-rescue artefact (§5.3) is protocol-specific.
6. **Burst-mode chaos.** Add a Poisson-burst fault scheduler to the rig and re-sweep the active roster at a single chaos rate. Hypothesis: the playbook helps more under burstiness because it makes recovery deterministic rather than emergent.
7. **A paired hardware-cost analysis** (joules per run, GPU-seconds per run) to complement the token/wall-time accounting in §4.7.

8. **Public dashboard.** All paper figures should be served live from the released CSVs so a reader can reslice the data interactively. This is a project-side deliverable rather than a paper one.

6.4 Closing remark

At 30 % chaos, a 69-word recovery playbook raised the success rate of the functioning models from 14.5 %–57.5 % to 56 %–97 % (86 %–97 % for all but the smallest of them), at a modest and predictable cost, and in our roster it never lowered an agent's success rate. Because each run is seeded and recorded, the same sweep that measures this also produces auditable evidence of how an agent behaves when its tools fail—the kind of evidence that both reliability engineering and the EU AI Act call for and that is rarely collected today. The rig, the playbooks and the 128,800-run dataset are released so that others can rerun, extend or refute these results.

Acknowledgements

An earlier prototype of this instrument—the *Chaos Playbook Engine*—won **first place in the Enterprise Agents track** of the Google × Kaggle *AI Agents Intensive* capstone competition (December 2025; 4,477 entries in the track among 11,500+ submissions overall). The prototype is open source at https://github.com/alberto-martinez-zurita/chaos-playbook-engine. The present study re-architected that prototype into the deterministic, protocol-agnostic A/B rig described in §3 and ran an entirely new experimental campaign; no result from the prototype is reused in this paper.

References

Chaos engineering and fault injection

[1] Basiri, A., Behnam, N., De Rooij, R., Hochstein, L., Kosewski, L., Reynolds, J., & Rosenthal, C. (2016). **Chaos engineering**. *IEEE Software*, 33(3), 35–41.

[2] Bennett, C., & Tseitlin, A. (2012, July 30). **Chaos Monkey released into the wild**. Netflix Tech Blog. Archived copy: https://web.archive.org/web/2012/http://techblog.netflix.com/2012/07/chaos-monkey-released-into-wild.html

[3] Chaos Community. (2019). **Principles of Chaos Engineering** (last updated March 2019). https://principlesofchaos.org

[4] Alvaro, P., Rosen, J., & Hellerstein, J. M. (2015). **Lineage-driven fault injection**. *ACM SIGMOD 2015*, 331–346.

[5] Alvaro, P., Andrus, K., Sanden, C., Rosenthal, C., Basiri, A., & Hochstein, L. (2016). **Automating failure testing research at internet scale**. *ACM Symposium on Cloud Computing (SoCC)*, 17–28.

[6] Gremlin Inc. (2024). **Gremlin Chaos Engineering Platform**. https://www.gremlin.com

[7] LitmusChaos contributors. (2024). **Litmus: Cloud-Native Chaos Engineering** (CNCF Incubating). https://litmuschaos.io

[8] Chaos Mesh contributors. (2024). **Chaos Mesh: A Cloud-Native Chaos Engineering Platform** (CNCF Incubating). https://chaos-mesh.org

LLM agent benchmarks and frameworks

[9] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). **ReAct: Synergizing reasoning and acting in language models**. *ICLR 2023*.

[10] Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., & Yao, S. (2023). **Reflexion: Language agents with verbal reinforcement learning**. *NeurIPS 2023*. arXiv:2303.11366.

[11] Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T. L., Cao, Y., & Narasimhan, K. (2023). **Tree of thoughts: Deliberate problem solving with large language models**. *NeurIPS 2023*.

[12] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E. H., Le, Q. V., & Zhou, D. (2022). **Chain-of-thought prompting elicits reasoning in large language models**. *NeurIPS 2022*.

[13] Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., & Narasimhan, K. (2024). **SWE-bench: Can language models resolve real-world GitHub issues?**. *ICLR 2024*.

[14] Liu, X., Yu, H., Zhang, H., Xu, Y., Lei, X., Lai, H., et al. (2024). **AgentBench: Evaluating LLMs as agents**. *ICLR 2024*. arXiv:2308.03688.

[15] Zhou, S., Xu, F. F., Zhu, H., Zhou, X., Lo, R., Sridhar, A., et al. (2024). **WebArena: A realistic web environment for building autonomous agents**. *ICLR 2024*.

[16] Mialon, G., Fourrier, C., Wolf, T., LeCun, Y., & Scialom, T. (2024). **GAIA: A benchmark for general AI assistants**. *ICLR 2024*. arXiv:2311.12983.

[17] Xie, T., Zhang, D., Chen, J., Li, X., Zhao, S., Cao, R., et al. (2024). **OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments**. *NeurIPS 2024 Datasets and Benchmarks Track*. arXiv:2404.07972.

[18] Yao, S., Shinn, N., Razavi, P., & Narasimhan, K. (2025). **τ-bench: A benchmark for tool-agent-user interaction in real-world domains**. *ICLR 2025*. arXiv:2406.12045.

Standard LLM benchmarks (for context)

[19] Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D., & Steinhardt, J. (2021). **Measuring massive multitask language understanding (MMLU)**. *ICLR 2021*.

[20] Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., et al. (2021). **Training verifiers to solve math word problems (GSM8K)**. arXiv:2110.14168.

[21] Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. de O., Kaplan, J., et al. (2021). **Evaluating large language models trained on code (HumanEval)**. arXiv:2107.03374.

Scaling, distillation, reasoning

[22] Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., et al. (2022). **An empirical analysis of compute-optimal large language model training** (Chinchilla). *NeurIPS 2022*. Preprint title: "Training compute-optimal large language models", arXiv:2203.15556.

[23] Hinton, G., Vinyals, O., & Dean, J. (2015). **Distilling the knowledge in a neural network**. *NIPS 2014 Deep Learning Workshop*. arXiv:1503.02531.

[24] Jiang, A. Q., Sablayrolles, A., Roux, A., Mensch, A., Savary, B., Bamford, C., et al. (2024). **Mixtral of experts**. arXiv:2401.04088.

Models used in this paper (technical reports / model cards)

[25] Qwen Team, Alibaba. (2026). **Qwen3.5: Towards Native Multimodal Agents** (0.8 B, 2 B, 4 B and 9 B models released 2026-03-02). https://qwen.ai/blog?id=qwen3.5

[26] Gemma Team, Google DeepMind. (2026). **Gemma 4 Technical Report**. arXiv:2607.02770. Model family released 2026-04-02 (E2B and E4B with Per-Layer Embeddings; 26 B-A4B MoE; 31 B dense).

[27] Zhipu AI / GLM Team. (2025). **GLM-4.5: Agentic, Reasoning, and Coding (ARC) Foundation Models** (355 B total / 32 B active MoE). arXiv:2508.06471.

[28] NVIDIA. (2026). **NVIDIA-Nemotron-3-Nano-4B model card** (released 2026-03-16): hybrid Mamba2–Transformer architecture, compressed from NVIDIA-Nemotron-Nano-9B-v2 using the Nemotron Elastic framework. https://huggingface.co/nvidia/NVIDIA-Nemotron-3-Nano-4B-GGUF

[29] Mistral AI. (2025). **Ministral 3 model family** (3 B / 8 B / 14 B; Base / Instruct / Reasoning variants), released December 2025. https://mistral.ai/news/mistral-3

[30] OpenAI. (2025). **gpt-oss-120b & gpt-oss-20b Model Card**. arXiv:2508.10925. https://openai.com/index/gpt-oss-model-card/

Protocols

[31] Anthropic. (2024). **Model Context Protocol (MCP) specification**. https://modelcontextprotocol.io/specification/latest

[32] Google. (2025). **Agent2Agent (A2A) protocol specification**. Announced April 9, 2025 at Google Cloud Next; donated to the Linux Foundation June 2025. https://a2a-protocol.org/latest/specification/

Compliance and robustness

[33] European Parliament & Council. (2024). **Regulation (EU) 2024/1689 on Artificial Intelligence (AI Act)**. https://eur-lex.europa.eu/eli/reg/2024/1689/oj

[34] Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., & Fritz, M. (2023). **Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection**. *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec '23)*, 79–90. doi:10.1145/3605764.3623985.

Statistical methods

[35] Wilson, E. B. (1927). **Probable inference, the law of succession, and statistical inference**. *Journal of the American Statistical Association*, 22(158), 209–212. doi:10.1080/01621459.1927.10502953.

[36] Newcombe, R. G. (1998). **Interval estimation for the difference between independent proportions: comparison of eleven methods**. *Statistics in Medicine*, 17(8), 873–890. Erratum: 18(10), 1293 (1999).

Concurrent work on LLM-agent reliability under chaos

[37] Gupta, A. (2026). **ReliabilityBench: Evaluating LLM Agent Reliability Under Production-Like Stress Conditions**. arXiv:2601.06112.

[38] Wang, A., Hager, S., Asija, A., Khashabi, D., & Andrews, N. (2025). **Hell or High Water: Evaluating Agentic Recovery from External Failures**. *COLM 2025*. arXiv:2508.11027.

[39] Wang, R., Chen, Y., Wang, Y., Wu, C., Fang, J., Cai, X., Gu, Q., Su, H., Zhang, A., Wang, X., Cai, X., & Chua, T.-S. (2026). **AgentNoiseBench: Benchmarking Robustness of Tool-Using LLM Agents Under Noisy Condition**. arXiv:2602.11348.

Inverted-U patterns in prompting

[40] Khan, I. (2025). **You Don't Need Prompt Engineering Anymore: The Prompting Inversion**. arXiv:2510.22251.

EU AI Act robustness evidence (Art. 15 / Art. 55)

[41] Bueno Momcilovic, T., Buesser, B., Zizzo, G., Purcell, M., & Balta, D. (2024). **Towards Assuring EU AI Act Compliance and Adversarial Robustness of LLMs**. arXiv:2410.05306.

[42] Nolte, H., Rateike, M., & Finck, M. (2025). **Robustness and Cybersecurity in the EU Artificial Intelligence Act**. *Proceedings of the 2025 ACM Conference on Fairness, Accountability, and Transparency (FAccT '25)*, 283–295. doi:10.1145/3715275.3732020. arXiv:2502.16184.

Recent work on tool-failure recovery

[43] Zhu, D., Ma, X., Shen, Y., Li, X., Zhao, Y., Wang, S., Yan, L., & Yin, D. (2026). **When Tools Fail: Benchmarking Dynamic Replanning and Anomaly Recovery in LLM Agents** (ToolMaze). arXiv:2606.05806.

[44] Mazumder, A., & Lia, N. J. (2026). **AgentCheck: A Reproduce-Intervene-Mitigate Workbench for LLM Agents over MCP**. arXiv:2607.11098.

Distillation and robustness (historical anchor)

[45] Papernot, N., McDaniel, P., Wu, X., Jha, S., & Swami, A. (2016). **Distillation as a Defense to Adversarial Perturbations against Deep Neural Networks**. *IEEE Symposium on Security and Privacy 2016*, 582–597. arXiv:1511.04508.

[46] Carlini, N., & Wagner, D. (2017). **Towards evaluating the robustness of neural networks**. *IEEE Symposium on Security and Privacy 2017*, 39–57. arXiv:1608.04644.